Υπολογιστικό Θέμα

Feedback Linearization for Double Pendulum and Planar Cepheus System under Model Uncertainty via Reinforcement Learning

Άγγελος Βόσσος
mc22039

Σεπτέμβριος 2026

Περιεχόμενα

Εισαγωγή **2**

1 Μοντέλο του Double Pendulum **3**
1.1 REINFORCE 12
1.2 PPO 20

2 Μοντέλο του Planar 2D Cepheus **26**
2.1 Κινηματική μοντελοποίηση 26
2.2 Δυναμική μοντελοποίηση 29
2.3 Feedback Linearization 30
2.4 Nominal μοντέλο 32
2.5 REINFORCE 33
2.6 PPO 37

3 Σύγκριση με το σενάριο του paper MED 2026 **40**
3.1 REINFORCE 41
3.2 PPO 43

Σχολιασμός αποτελεσμάτων και Συμπεράσματα **45**

Βιβλιογραφία **46**

Εισαγωγή

Το feedback linearization αποτελεί μία μέθοδο μη γραμμικού ελέγχου, μέσω της οποίας μπορούμε να μετατρέψουμε τη δυναμική ενός μη γραμμικού συστήματος σε μία γραμμική σχέση της εισόδου με την έξοδο, επιλέγοντας κατάλληλα την είσοδο ελέγχου με στόχο να αναιρεθούν οι μη γραμμικοί όροι του συστήματος [1]. Για να είναι αποτελεσματικός ο έλεγχος μέσω αυτής της μεθόδου είναι απαραίτητη η ακριβής γνώση του δυναμικού μοντέλου. Στην πραγματικότητα όμως, τα δυναμικά μοντέλα σπάνια είναι πλήρως γνωστά, με αποτέλεσμα να μην είναι δυνατός ο έλεγχος με κλασικό feedback linearization στα συστήματα αυτά.

Στο paper των Westenbroek et al. [2] προτείνεται η χρήση reinforcement learning για την εύρεση διορθωτικών όρων, οι οποίοι προστίθενται στον feedback linearizing controller και μέσω κατάλληλης εκπαίδευσης μειώνουν την επίδραση των αποκλίσεων από την πραγματική δυναμική και επιτρέπουν την ορθή παρακολούθηση τροχιάς για το σύστημα.

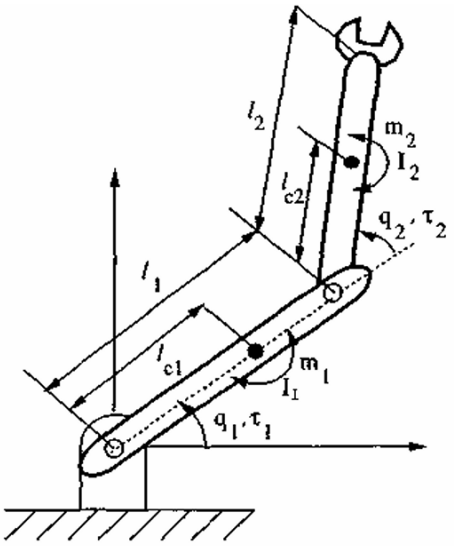
Στην παρούσα εργασία επιχειρήθηκε η υλοποίηση της μεθόδου που αναπτύχθηκε στο [2] για αρχικά ένα αρκετά απλό σύστημα, το fully actuated double pendulum, το οποίο ήταν από τα συστήματα που μελετήθηκαν και στο paper. Το δυναμικό μοντέλο λήφθηκε από τους Slotine και Li [1] και θεωρήθηκαν διαφορετικές παράμετροι για το πραγματικό και το nominal μοντέλο και έγιναν δοκιμές για την εκμάθηση των διορθωτικών όρων μέσω nonlinear features. Αρχικά εξετάστηκε η βελτιστοποίηση των παραμέτρων με τη χρήση του MATLAB, ενώ στη συνέχεια εφαρμόστηκαν οι αλγόριθμοι REINFORCE [3] και Proximal Policy Optimization (PPO) [4].

Έπειτα, η μέθοδος εφαρμόστηκε και στο Planar 2D Cepheus το οποίο παρουσιάστηκε στο paper των Skandalakis et al. (MED 2026) [5]. Αρχικά, πραγματοποιήθηκε η εξαγωγή του δυναμικού μοντέλου μέσω της κινηματικής και των εξισώσεων Euler-Lagrange και έπειτα εφαρμόστηκαν ξανά οι μέθοδοι REINFORCE και PPO για την εύρεση των διορθωτικών όρων του feedback linearization controller. Η μέθοδος υλοποιήθηκε για nominal μοντέλο με αρκετά μεγάλες αποκλίσεις από το πραγματικό, αλλά και για το nominal μοντέλο που παρουσιάστηκε στο [5], ώστε να πραγματοποιηθεί το ίδιο task που αφορούσε trajectory tracking και να συγκριθούν τα αποτελέσματα σε σχέση με αυτά του paper.

Οι εκπαιδευμένοι ελεγκτές γενικά αξιολογήθηκαν σε διαφορετικά tasks, με σκοπό να φανεί η αποτελεσματικότητά τους, αλλά και η δυνατότητα γενίκευσης τους σε διαφορετικές επιθυμητές τροχιές.

1 Μοντέλο του Double Pendulum

Ως πρώτο σύστημα μελέτης χρησιμοποιήθηκε ένα πλήρως actuated double pendulum από το κεφάλαιο 6 του Slotine and Li [1].



Σχήμα 1: Fully actuated double pendulum (Slotine and Li, 1991).

Το δυναμικό μοντέλο του συστήματος γράφεται στη μορφή:

$$M(q)\ddot{q} + c(q,\dot{q}) + G(q) = u. \tag{1}$$

όπου

$$M(q) = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \tag{2}$$

$$M_{11} = m_1 l_{c1}^2 + I_1 + m_2\left(l_1^2 + l_{c2}^2 + 2l_1 l_{c2}\cos\theta_2\right) + I_2,$$

$$M_{12} = M_{21} = m_2\left(l_{c2}^2 + l_1 l_{c2}\cos\theta_2\right) + I_2, \tag{3}$$

$$M_{22} = m_2 l_{c2}^2 + I_2$$

Ο όρος Coriolis και φυγόκεντρων δυνάμεων είναι:

$$c(q,\dot{q}) = \begin{bmatrix} -m_2 l_1 l_{c2} \sin(\theta_2)\left(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2\right) \\ \\ m_2 l_1 l_{c2} \sin(\theta_2)\dot{\theta}_1^2 \end{bmatrix} \tag{4}$$

ενώ ο όρος της βαρύτητας δίνεται από:

$$G(q) = \begin{bmatrix} (m_1 l_{c1} + m_2 l_1) g \cos(\theta_1) + m_2 l_{c2} g \cos(\theta_1 + \theta_2) \\ \\ m_2 l_{c2} g \cos(\theta_1 + \theta_2) \end{bmatrix} \tag{5}$$

Οι γενικευμένες συντεταγμένες, οι μεταβλητές κατάστασης και η είσοδος ορίζονται αντίστοιχα ως:

$$\tilde{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \qquad \tilde{x} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}, \qquad u = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \tag{6}$$

Η αναπαράσταση του συστήματος στον χώρο κατάστασης είναι:

$$\dot{\tilde{x}} = \begin{bmatrix} \dot{\tilde{q}} \\ M(\tilde{q})^{-1}\left(u - c(\tilde{q},\dot{\tilde{q}}) - G(\tilde{q})\right) \end{bmatrix}. \tag{7}$$

Συγκεκριμένα, θεωρήθηκε ένα μοντέλο ως το πραγματικό, με παραμέτρους:

$$\begin{gathered} m_1 = m_2 = 1\,\mathrm{kg}, \\ l_1 = l_2 = 1\,\mathrm{m}, \\ l_{c1} = \frac{l_1}{2}, \qquad l_{c2} = \frac{l_2}{2} \end{gathered} \tag{8}$$

Και ένα «λανθασμένο» (nominal) μοντέλο με παραμέτρους:

$$\begin{gathered} m_{1,\mathrm{nom}} = \frac{m_1}{3}, \qquad m_{2,\mathrm{nom}} = \frac{m_2}{3}, \\ l_{1,\mathrm{nom}} = \frac{l_1}{3}, \qquad l_{2,\mathrm{nom}} = \frac{l_2}{3}, \\ l_{c1,\mathrm{nom}} = \frac{l_{1,\mathrm{nom}}}{2}, \qquad l_{c2,\mathrm{nom}} = \frac{l_{2,\mathrm{nom}}}{2} \end{gathered} \tag{9}$$

Στο paper ο εκτιμώμενος feedback linearizing controller γράφεται στη γενική μορφή:

$$\hat{u}_\theta(x, v) = [\beta_m(x) + \beta_{\theta_1}(x)] + [\alpha_m(x) + \alpha_{\theta_2}(x)]\, v \tag{10}$$

Χρησιμοποιήθηκε καταρχάς μία λίγο πιο απλοποιημένη εκδοχή της ίδιας ιδέας, βάζοντας έναν ενιαίο, αντί για δύο, διορθωτικό όρο πάνω στο nominal feedback linearization:

$$u = u_{\text{nominal}} + u_{\text{learned}} \tag{11}$$

Στο paper τονίζεται πως οι διορθώσεις είναι καλό να γράφονται στη μορφή:

$$\beta_{\theta_1}(x) = \sum_{k=1}^{K_1} \theta_1^k \beta_k(x), \qquad \alpha_{\theta_2}(x) \;\; = \sum_{k=1}^{K_2} \theta_2^k \alpha_k(x) \tag{12}$$

όπου τα $\beta_k(x)$ και $\alpha_k(x)$ είναι συνεχείς συναρτήσεις, που ονομάζονται nonlinear features. Κατά αυτόν τον τρόπο οι διορθωτικοί όροι παραμετροποιούνται ως γραμμικοί συνδυασμοί μη γραμμικών χαρακτηριστικών, ώστε εν τέλει η εξάρτηση από τις άγνωστες παραμέτρους να παραμένει γραμμική.

Συνεπώς, ο όρος u_{learned} στον controller επιλέχθηκε να είναι της μορφής:

$$u_{\text{learned}} = \theta \cdot \phi(x, v) \tag{13}$$

Στο διάνυσμα $\phi(x, v)$ χρησιμοποιήθηκαν ορισμένοι τριγωνομετρικοί όροι, οι οποίοι κυριαρχούν στο μοντέλο του double pendulum, καθώς και οι μεταβλητές κατάστασης.

$$\phi(x,v) = \begin{bmatrix} 1 \\ \sin(\theta_1) \\ \cos(\theta_1) \\ \sin(\theta_2) \\ \cos(\theta_2) \\ \sin(\theta_1+\theta_2) \\ \cos(\theta_1+\theta_2) \\ \dot{\theta}_1 \\ \dot{\theta}_2 \\ v_1 v_2 \end{bmatrix} \tag{14}$$

Ορίστηκε ένα τυχαίο desired trajectory:

$$q_d = \begin{bmatrix} \theta_{1,d} \\ \theta_{2,d} \end{bmatrix} = \begin{bmatrix} 0.5\sin(t) \\ 0.5\cos(t) \end{bmatrix} \tag{15}$$

Με αρχικές συνθήκες:

$$x_0 = \begin{bmatrix} \pi/4 & \pi/6 & 0 & 0 \end{bmatrix} \tag{16}$$

Για τον υπολογισμό των παραμέτρων θ του διορθωτικού όρου u_{learned}, υλοποιήθηκε αρχικά ένα απλό πρόβλημα βελτιστοποίησης, ορίζοντας ως συνάρτηση κόστους:

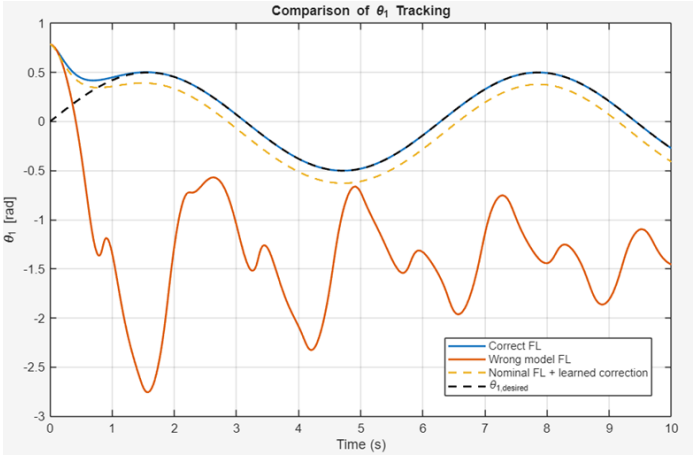
$$J = \int_0^{t_{\text{end}}} \|e(t)\|^2 \, dt + 10^{-4}\|\theta\|^2 \tag{17}$$

με:

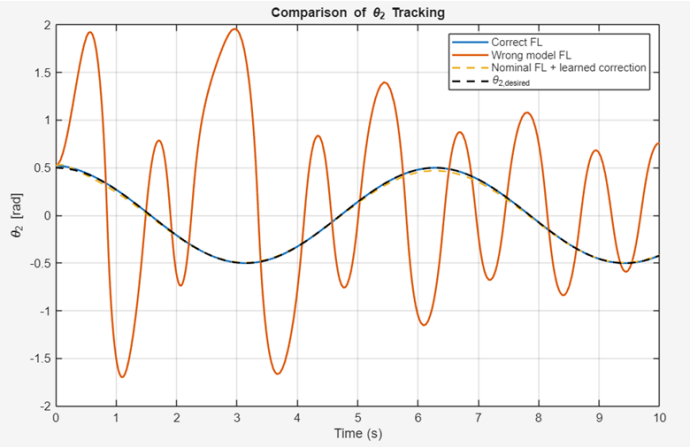
$$e(t) = q(t) - q_d(t) \tag{18}$$

Ο δεύτερος όρος χρησιμοποιήθηκε ώστε να αποφεύγονται πολύ μεγάλες τιμές των βαρών θ.

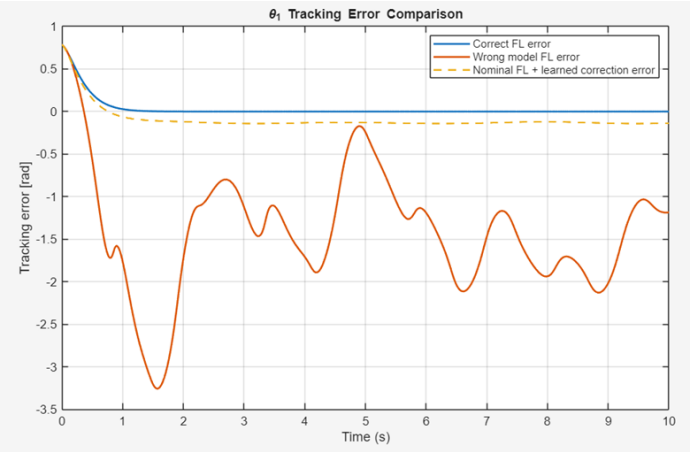
Για την επίλυση του προβλήματος βελτιστοποίησης χρησιμοποιήθηκε η εντολή `patternsearch` στο MATLAB. Με FL εννοείται το feedback linearization.



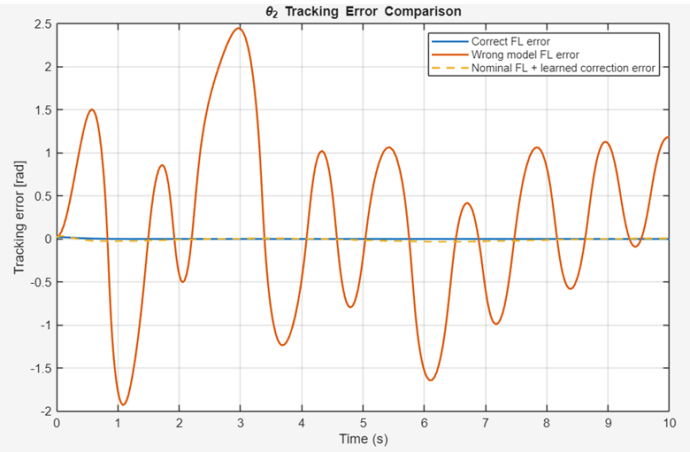
(α′) Απόκριση της γωνίας θ_1.



(β′) Απόκριση της γωνίας θ_2.



(γ′) Σφάλμα παρακολούθησης της θ_1.



(δ′) Σφάλμα παρακολούθησης της θ_2.

Σχήμα 2: Αποτελέσματα της αρχικής βελτιστοποίησης του controller

Μπλε καμπύλη (Correct FL): παρουσιάζεται η εφαρμογή feedback linearization με γνώση του πραγματικού μοντέλου. Ο έλεγχος είναι πρακτικά ιδανικός και η απόκριση του συστήματος ακολουθεί με μεγάλη ακρίβεια την επιθυμητή τροχιά.

Πορτοκαλί καμπύλη (Wrong model FL): παρουσιάζεται η εφαρμογή feedback linearization χρησιμοποιώντας μόνο το λανθασμένο nominal μοντέλο, με αποτέλεσμα να εμφανίζονται έντονες αποκλίσεις από την επιθυμητή τροχιά.

Κίτρινη διακεκομμένη καμπύλη (Nominal FL + learned correction): παρουσιάζεται ο έλεγχος που προκύπτει από την εφαρμογή της προηγούμενης μεθόδου, όπου είναι γνωστό μόνο το nominal μοντέλο, αλλά χρησιμοποιείται επιπλέον ο learned διορθωτικός όρος. Παρατηρείται σημαντική μείωση της απόκλισης και η απόκριση ακολουθεί σε μεγάλο βαθμό τη μορφή της επιθυμητής τροχιάς. Παρ' όλα αυτά, για τη γωνία θ_1 παραμένει εμφανές ένα σταθερό offset.

Στη συνέχεια, ο υπολογισμός του διορθωτικού όρου u_{learned} πραγματοποιήθηκε με μία σχετικά απλοποιημένη προσέγγιση reinforcement learning.

Η εκπαίδευση πραγματοποιήθηκε με μία μέθοδο τύπου rollout-based policy search. Αρχικά οι παράμετροι τέθηκαν ίσες με μηδέν, $\theta = 0$, και σε κάθε epoch δημιουργούνταν τυχαίες διαταραχές ε_i των παραμέτρων. Για κάθε διαταραχή αξιολογούνταν δύο κοντινές λύσεις:

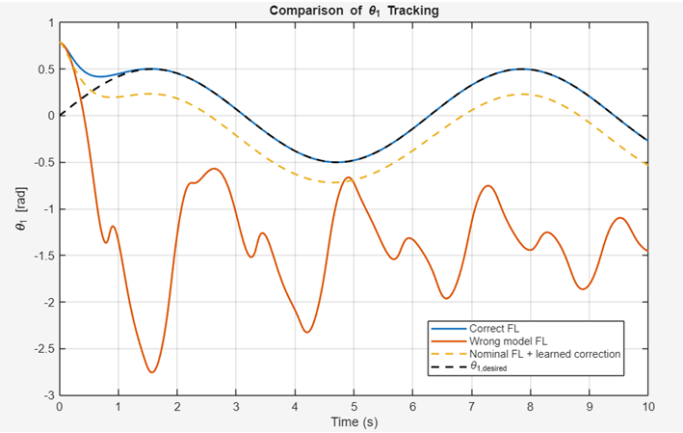
$$\theta^+ = \theta + \sigma\varepsilon_i, \qquad \theta^- = \theta - \sigma\varepsilon_i \tag{19}$$

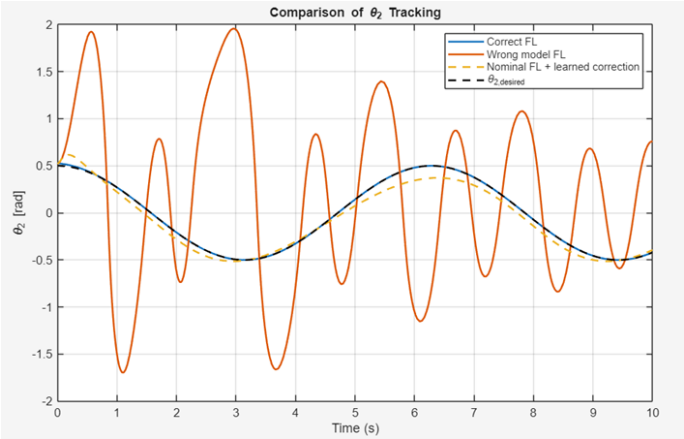
Για καθεμία από τις δύο λύσεις υπολογιζόταν το κόστος J και από τη διαφορά των δύο τιμών εκτιμούταν η κατεύθυνση βελτίωσης:

$$g \approx \frac{J^- - J^+}{2\sigma}\varepsilon_i \tag{20}$$

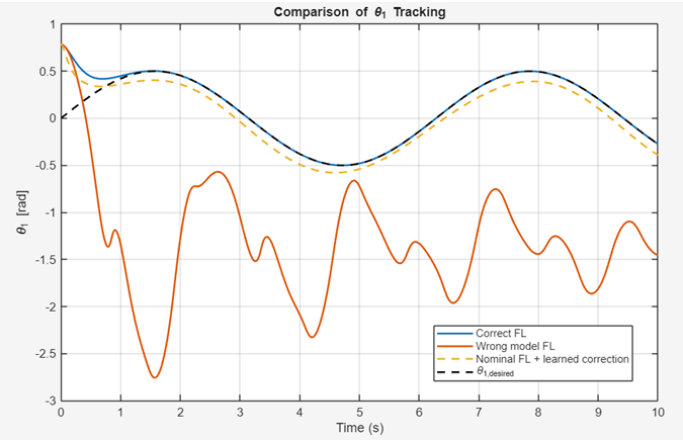
Στη συνέχεια πραγματοποιούταν η ενημέρωση των παραμέτρων:

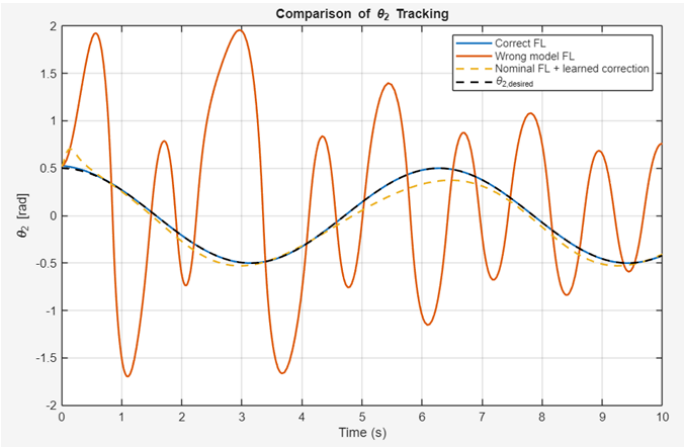
$$\theta \leftarrow \theta + \alpha g \tag{21}$$



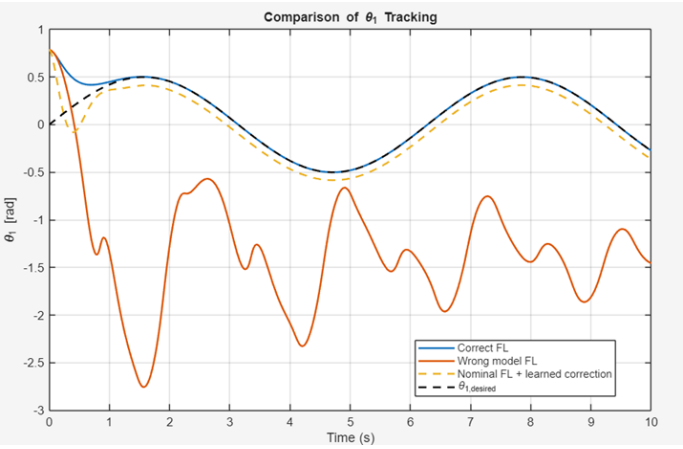


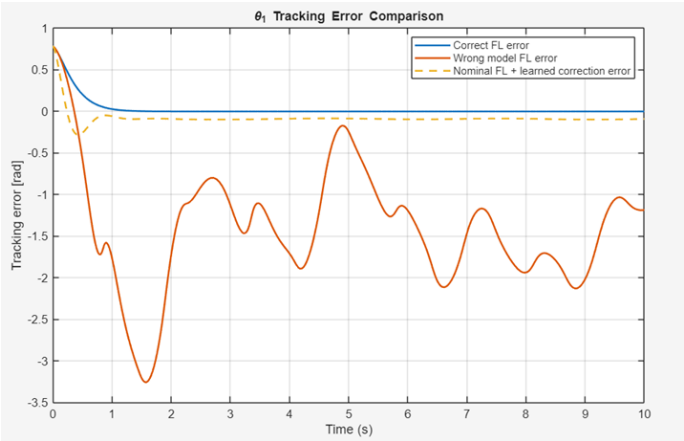
Αποτελέσματα για 1000 epochs





Αποτελέσματα για 30000 epochs





Αποτελέσματα για $K_p = [100, 25]$

Σχήμα 3: Αποτελέσματα για Central-Difference Rollout-Based Policy Search

Επειδή παρέμενε η απόκλιση στην απόκριση του συστήματος, υλοποιήθηκε στη συνέχεια η πιο γενική σχέση για τον learned feedback linearizing controller:

$$\hat{u}_\theta(x, v) = [\beta_m(x) + \beta_{\theta_1}(x)] + [\alpha_m(x) + \alpha_{\theta_2}(x)]\, v \tag{22}$$

όπου:

$$\beta_m(x) = \hat{C}(q, \dot{q})\dot{q} + \hat{G}(q) \tag{23}$$

$$\alpha_m(x) = \hat{M}(q) \tag{24}$$

Συνεπώς, ο learned feedback linearizing controller παίρνει τη μορφή:

$$\hat{u}_\theta(x, v) = \left[\hat{C}(q, \dot{q})\dot{q} + \hat{G}(q) + \Delta\beta(x)\right] + \left[\hat{M}(q) + \Delta\alpha(x)\right] v \tag{25}$$

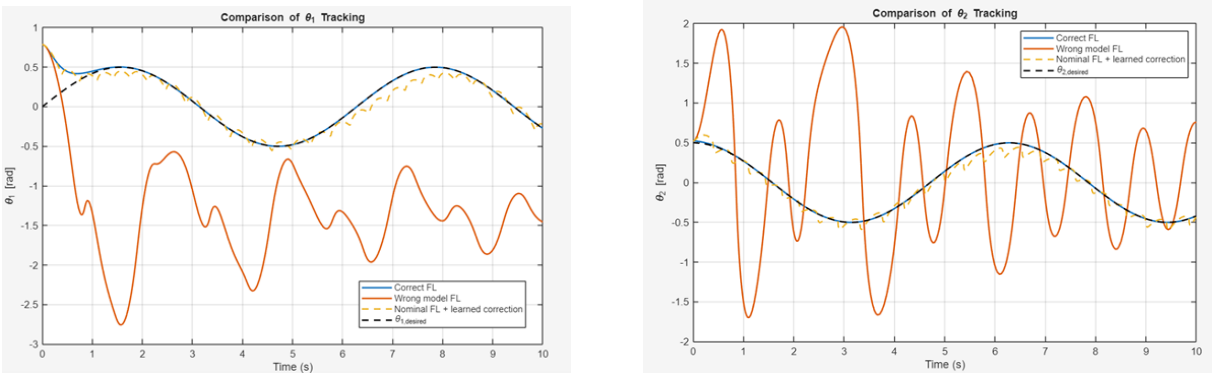
Για τους δύο διορθωτικούς όρους χρησιμοποιήθηκε η παραμετροποίηση με nonlinear features που παρουσιάστηκε προηγουμένως. Η μορφή του όρου $\Delta\alpha(x)$ είναι:

$$\Delta\alpha(x) = \begin{bmatrix} \alpha_{11}(x) & \alpha_{12}(x) \\ \alpha_{21}(x) & \alpha_{22}(x) \end{bmatrix} \tag{26}$$

όπου κάθε στοιχείο του πίνακα παραμετροποιείται ως γραμμικός συνδυασμός nonlinear features, σύμφωνα με τη γενική μορφή:

$$\alpha_{\theta_2}(x) = \sum_{k=1}^{K_2} \theta_2^k \alpha_k(x) \tag{27}$$

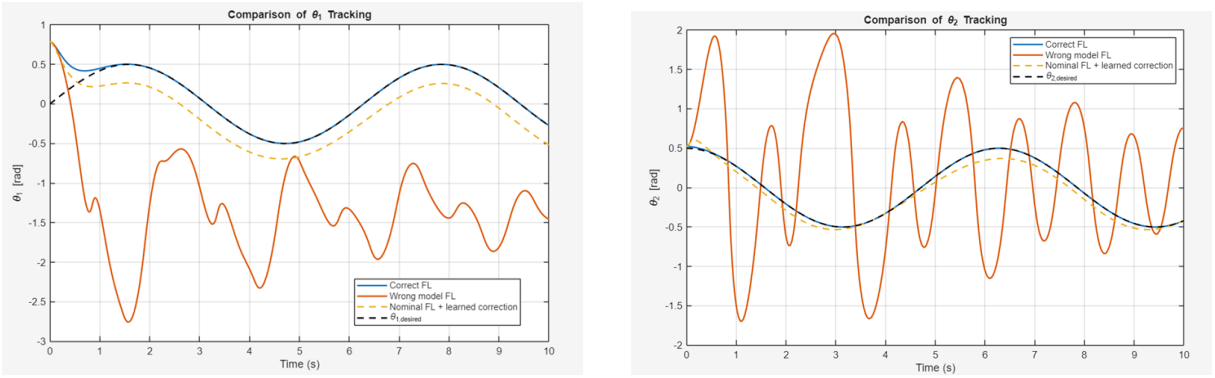
Αρχικά χρησιμοποιήθηκε πολύ μεγάλη πληθώρα nonlinear features και, με την ίδια απλοποιημένη μέθοδο reinforcement learning, πραγματοποιήθηκε εκπαίδευση για περίπου 400 epochs.



Σχήμα 4: Αποτελέσματα για πολύ μεγάλη πληθώρα nonlinear features

Λόγω της χρήσης πολύ μεγάλης πληθώρας nonlinear features, παρατηρήθηκε έντονη ταλαντωτική συμπεριφορά στις αποκρίσεις του συστήματος, παρόλο που το offset για τη γωνία θ_1 φάνηκε στη συγκεκριμένη περίπτωση να έχει περιοριστεί σημαντικά.

Στη συνέχεια χρησιμοποιήθηκαν συνολικά 21 nonlinear features (συνολικά 70 παραμέτρους), για τους δύο διορθωτικούς όρους $\Delta\alpha(x)$ και $\Delta\beta(x)$, και πραγματοποιήθηκε εκπαίδευση για 500 epochs.



Σχήμα 5: Αποτελέσματα για 21 nonlinear features και 500 epochs

Από τα αποτελέσματα φαίνεται ότι η απόκριση του συστήματος μοιάζει αρκετά με τις προηγούμενες αρχικές περιπτώσεις. Για τη γωνία θ_1 εμφανίζεται ξανά έντονο offset, παρόλο που η μορφή της απόκρισης παραμένει παρόμοια με αυτή της επιθυμητής τροχιάς.

Συνεπώς, το μόνο που φάνηκε θεωρητικά να συμβάλλει σημαντικά στη μείωση του offset ήταν η αύξηση του K_p.

1.1 REINFORCE

Για την εκπαίδευση του controller εφαρμόστηκε στη συνέχεια ο αλγόριθμος REINFORCE. Το policy ορίζεται ως:

$$\pi_\theta(u \mid x, v) \tag{28}$$

όπου το state του συστήματος είναι:

$$\tilde{x} = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}, \tag{29}$$

ενώ v είναι η εικονική είσοδος και u η πραγματική είσοδος του συστήματος, δηλαδή το action.

Κατά την εκπαίδευση επιλέγονται διαφορετικές είσοδοι u, δηλαδή διαφορετικά actions, σύμφωνα με Gaussian κατανομή. Η μέση τιμή της κατανομής καθορίζεται από τον feedback linearizing controller:

$$\mu_\theta(x, v) = [\beta_m(x) + \beta_\theta(x)] + [\alpha_m(x) + \alpha_\theta(x)]\, v \tag{30}$$

Κατά την εκπαίδευση προστίθεται θόρυβος στη μέση τιμή:

$$u = \mu_\theta(x, v) + \sigma\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I) \tag{31}$$

Επομένως, το policy είναι στοχαστικό και βασίζεται σε Gaussian κατανομή:

$$\pi_\theta(u \mid x, v) = \mathcal{N}\left(\mu_\theta(x, v), \sigma^2 I\right) \tag{32}$$

Αν μία δειγματοληπτημένη είσοδος, δηλαδή ένα action u, οδηγήσει σε καλύτερο αποτέλεσμα, ο αλγόριθμος αυξάνει την πιθανότητα να επιλεγεί στο μέλλον παρόμοια είσοδος. Αντίστοιχα, αν το αποτέλεσμα είναι χειρότερο, η πιθανότητα μειώνεται. Για τον σκοπό αυτό χρησιμοποιείται ο όρος:

$$\nabla_\theta \log \pi_\theta(u \mid x, v) \tag{33}$$

Ιδανικά, όπως αναφέρεται και στο paper, ζητείται η εύρεση των παραμέτρων θ έτσι ώστε, για κάθε state x και κάθε εικονική είσοδο v:

$$W_\theta(x, v) = \ddot{q} \approx v \tag{34}$$

Ορίζεται τότε η συνάρτηση:

$$\ell(x, v, \theta) = \|v - \ddot{q}\|_2^2 \tag{35}$$

Σκοπός είναι η ελαχιστοποίηση του παραπάνω όρου και συνεπώς το reward ορίζεται ως:

$$r = -\ell(x, v, \theta) = -\|v - \ddot{q}\|_2^2 \tag{36}$$

Η συνάρτηση reward υλοποιήθηκε επίσης σε quadratic form, ώστε να μπορούν να μεταβάλλονται ανεξάρτητα τα βάρη των όρων $(\ddot{q}_i - v_i)$. Η μορφή της είναι:

$$r_k = -(\ddot{q}_k - v_k)^T Q (\ddot{q}_k - v_k) \Delta t \tag{37}$$

όπου Q είναι το μητρώο βαρών, το οποίο αρχικά τέθηκε ίσο με το μοναδιαίο μητρώο. Η μεταβολή των βαρών χρησιμοποιήθηκε αργότερα ώστε να εξεταστεί αν μπορεί να περιοριστεί το offset στη γωνία θ_1.

Το στιγμιαίο reward r_k αξιολογεί το αποτέλεσμα του action u_k μόνο στο συγκεκριμένο χρονικό βήμα. Ωστόσο, το action μεταβάλλει το επόμενο state του συστήματος και επομένως μπορεί να επηρεάσει και τα μελλοντικά rewards του rollout. Για τον λόγο αυτό, για κάθε χρονικό βήμα k ορίζεται το return:

$$G_k = \sum_{j=k}^{N-1} \gamma^{j-k} r_j = r_k + \gamma r_{k+1} + \gamma^2 r_{k+2} + \cdots + \gamma^{N-1-k} r_{N-1} \tag{38}$$

όπου N είναι ο αριθμός των χρονικών βημάτων του rollout και γ ο discount factor, ο οποίος τέθηκε ίσος με:

$$\gamma = 1 \tag{39}$$

Το G_k εκφράζει επομένως το συνολικό reward που προκύπτει από το χρονικό βήμα k έως το τέλος του rollout.

Το return κάθε action συγκρίνεται με ένα baseline. Για το χρονικό βήμα k, το baseline υπολογίζεται ως ο μέσος όρος των returns των N_r rollouts:

$$b_k = \frac{1}{N_r} \sum_{i=1}^{N_r} G_k^{(i)} \tag{40}$$

Η χρήση του baseline μειώνει τη διακύμανση του policy gradient estimator και συμβάλλει σε πιο σταθερή εκπαίδευση.

Ορίζεται το advantage για το rollout i και το χρονικό βήμα k ως:

$$A_k^{(i)} = G_k^{(i)} - b_k \tag{41}$$

Αν $A_k^{(i)} > 0$, το συγκεκριμένο action έδωσε καλύτερο return από τον μέσο όρο, ενώ αν $A_k^{(i)} < 0$, έδωσε χειρότερο αποτέλεσμα.

Συνεπώς, ο όρος που χρησιμοποιείται για την ενημέρωση του policy γράφεται ως:

$$\nabla_\theta \log \pi_\theta \left(u_k^{(i)} \mid x_k^{(i)}, v_k^{(i)} \right) A_k^{(i)} \tag{42}$$

Για N_r rollouts, ο policy gradient estimator υπολογίζεται από τον μέσο όρο όλων των χρονικών βημάτων και όλων των rollouts σε κάθε epoch:

$$\nabla_\theta J(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \sum_{k=0}^{N-1} \nabla_\theta \log \pi_\theta \left(u_k^{(i)} \mid x_k^{(i)}, v_k^{(i)} \right) A_k^{(i)} \tag{43}$$

Για Gaussian policy, ο gradient του λογαρίθμου του policy δίνεται από:

$$\nabla_\theta \log \pi_\theta(u \mid x, v) = J_{\mu_\theta}(x, v)^T \frac{u - \mu_\theta(x, v)}{\sigma^2} \tag{44}$$

Ο όρος

$$u - \mu_\theta(x, v) \tag{45}$$

εκφράζει την κατεύθυνση της εξερεύνησης σε σχέση με τη μέση τιμή του policy.

Το Jacobian της μέσης τιμής του policy ως προς τις παραμέτρους ορίζεται ως:

$$J_{\mu,k} = \frac{\partial \mu_\theta(x_k, v_k)}{\partial \theta} \tag{46}$$

Τελικά, η ενημέρωση των παραμέτρων του controller πραγματοποιείται σύμφωνα με:

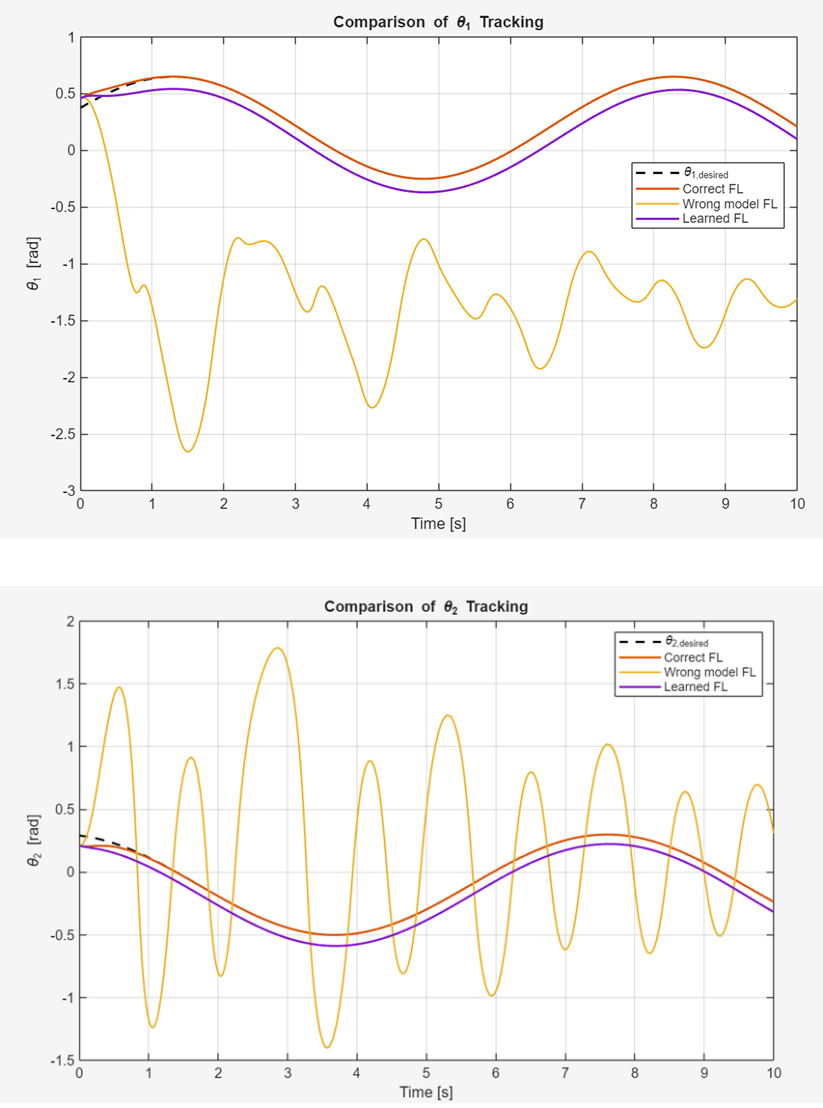
$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \tag{47}$$

Ακολουθώντας τη διαδικασία του paper, σε κάθε epoch πραγματοποιούνται $N_r = 50$ rollouts, με το καθένα να έχει χρονική διάρκεια $T = 0.25$ s και χρονικό βήμα $\Delta t = 0.01$ s, δηλαδή 25 χρονικά βήματα ανά rollout.

Για 15000 epochs και με τον πίνακα βαρών Q να δίνει ίδιο βάρος και στις δύο συνιστώσες του linearization error, τα αποτελέσματα ήταν αρκετά όμοια με τα προηγούμενα, καθώς παρέμενε εμφανές ένα offset στη γωνία θ_1.

Εφαρμόζοντας όμως τον αλγόριθμο REINFORCE, πάλι για 15000 epochs, αλλά αυτή τη φορά με το Q να δίνει διπλάσια βαρύτητα στο linearization error της πρώτης άρθρωσης, χρησιμοποιήθηκε:

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \tag{48}$$

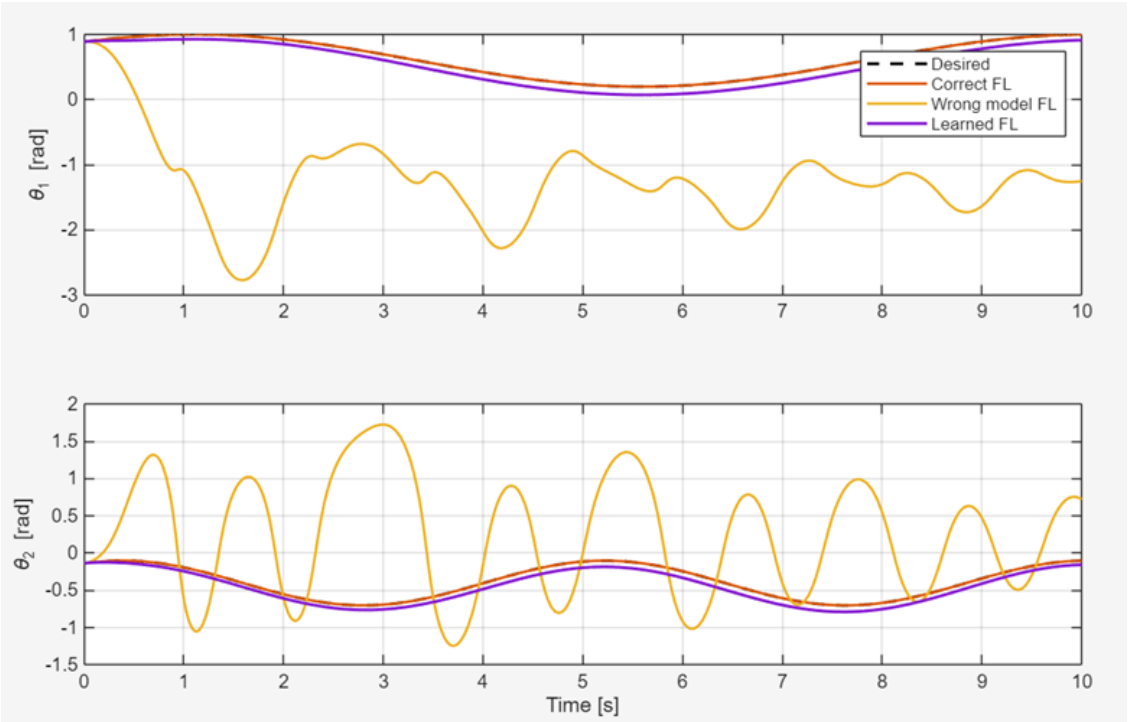


Σχήμα 6: Αποτελέσματα REINFORCE για 15000 epochs

Για:

$$q_{1,d}(t) = 0.6 + 0.4\sin(0.7t + 0.8), \tag{49}$$

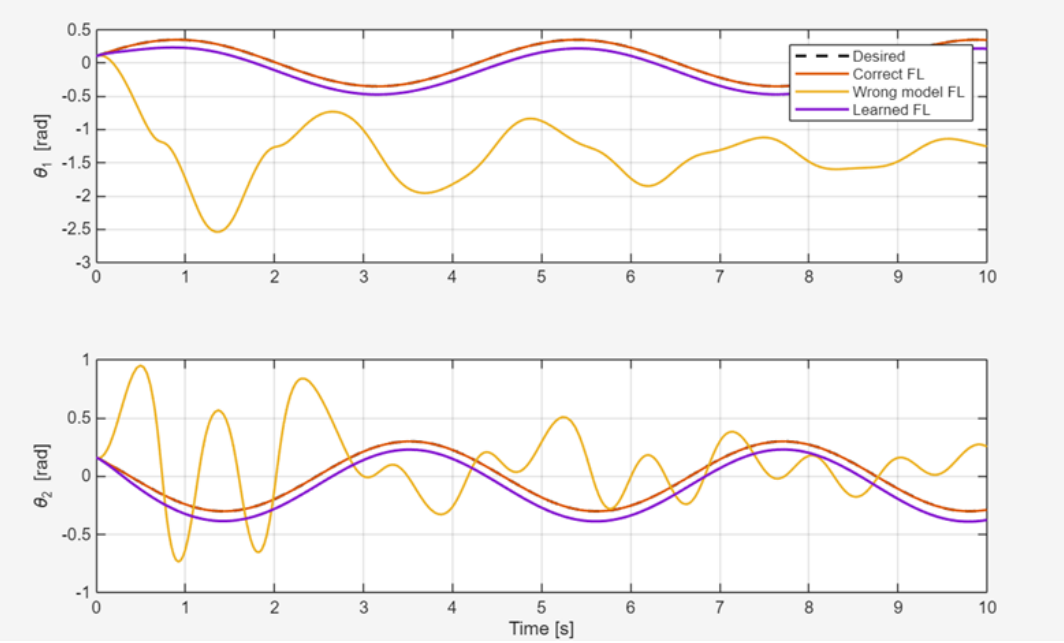
$$q_{2,d}(t) = -0.4 + 0.3\cos(1.3t - 0.5) \tag{50}$$



Για:

$$q_{1,d}(t) = 0.35\sin(1.4t + 0.3), \tag{51}$$

$$q_{2,d}(t) = 0.30\cos(1.5t + 1) \tag{52}$$

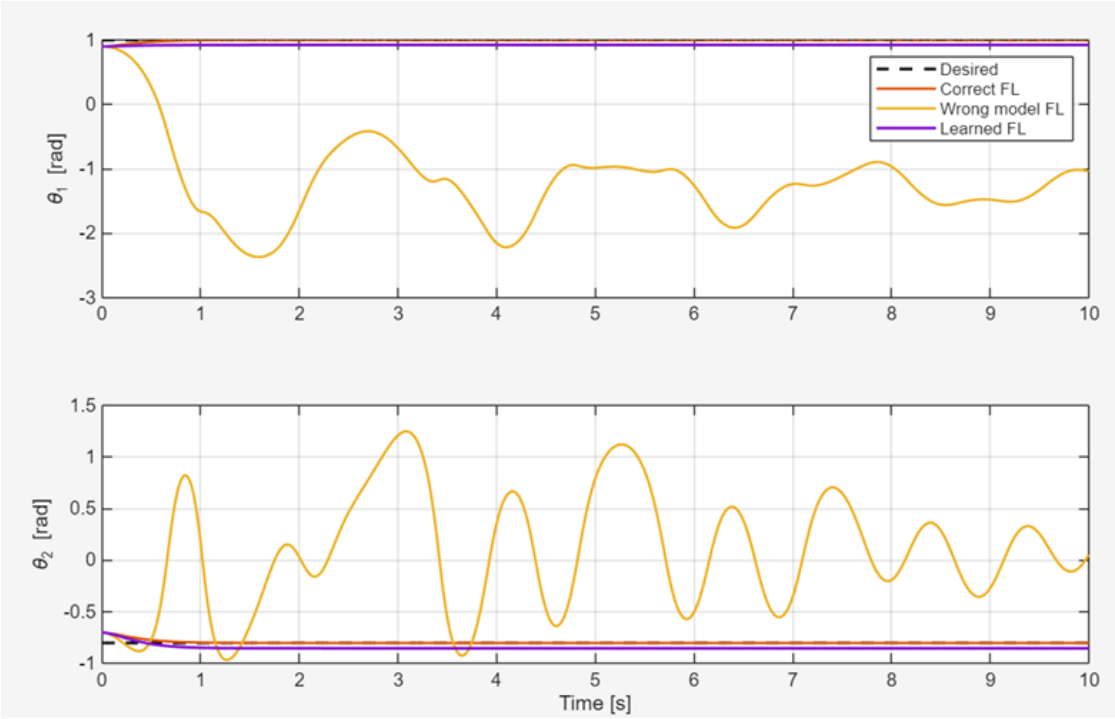


Για:

$$q_d(t) = \begin{bmatrix} 1 \\ -0.8 \end{bmatrix} \tag{53}$$

Με:

$$x(0) = [0.9 \quad -0.7 \quad 0 \quad 0] = [\quad q \quad \dot{q} \quad] \tag{54}$$



Για trajectory:

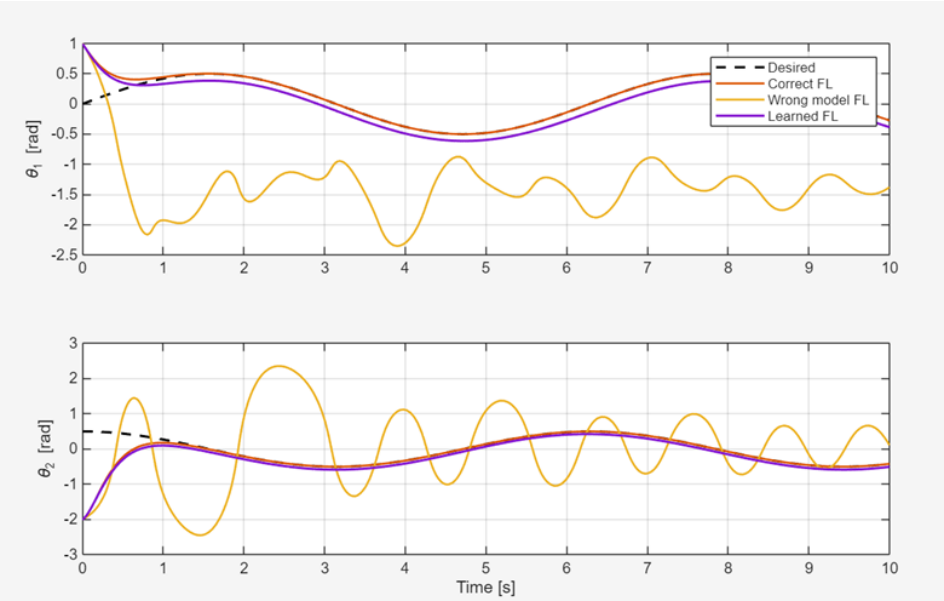
$$q_{1,d}(t) = 0.5\sin(t) \tag{55}$$

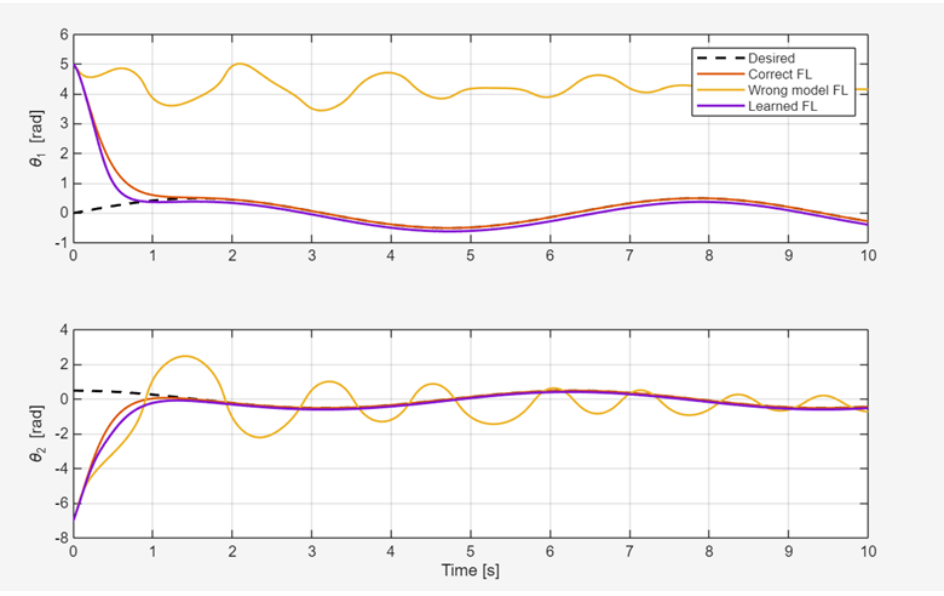
$$q_{2,d}(t) = 0.5\cos(t) \tag{56}$$

Και δύο περιπτώσεις «κακών» αρχικών συνθηκών, αντίστοιχα:

$$x(0) = [1 \quad -2 \quad -2 \quad 1.3] \tag{57}$$

$$x(0) = [5 \quad -7 \quad -2 \quad 12] \tag{58}$$





1.2 PPO

Η μέθοδος PPO μοιάζει σε μεγάλο βαθμό με τη REINFORCE, όμως έχει ως βασική διαφορά την τάση να περιορίζει το μέγεθος της αλλαγής της πολιτικής σε κάθε ενημέρωση, ώστε να αποφεύγονται πολύ απότομες και έντονες μεταβολές των παραμέτρων.

Ο βασικός στόχος παραμένει η όσο το δυνατόν καλύτερη πραγματοποίηση του feedback linearization, δηλαδή η επιτάχυνση του συστήματος να ακολουθεί όσο το δυνατόν καλύτερα την επιθυμητή εικονική είσοδο:

$$\ddot{q} \approx v \tag{59}$$

Όπως και στη REINFORCE, χρησιμοποιείται Gaussian policy:

$$\pi_\theta(u \mid x, v) = \mathcal{N}\left(\mu_\theta(x, v), \sigma^2 I\right) \tag{60}$$

με μέση τιμή:

$$\mu_\theta(x, v) = [\beta_m(x) + \beta_\theta(x)] + [\alpha_m(x) + \alpha_\theta(x)]\, v \tag{61}$$

Στην αρχή κάθε iteration αποθηκεύονται οι τρέχουσες παράμετροι ως παράμετροι της παλιάς πολιτικής:

$$\theta_{\text{old}} \leftarrow \theta \tag{62}$$

Τα actions που χρησιμοποιούνται για τη συλλογή των rollouts δειγματοληπτούνται από την παλιά πολιτική:

$$u_k \sim \pi_{\theta_{\text{old}}}(u_k \mid x_k, v_k) \tag{63}$$

Τα returns G_k υπολογίζονται με τον ίδιο τρόπο όπως στη REINFORCE, ως Monte Carlo returns.

Παρ' όλα αυτά, το baseline b_k υπολογίζεται μέσω μίας value function, η οποία αποτελεί τον critic:

$$V_w(x_k, v_k, k) = w^T \phi_V(x_k, v_k, k) \tag{64}$$

Το διάνυσμα χαρακτηριστικών του critic περιγράφει την κατάσταση του συστήματος, την εικονική είσοδο και τον εναπομένοντα χρόνο του rollout:

$$\phi_V = \begin{bmatrix} 1 \\ z_V \\ \{z_{V,i} z_{V,j}\}_{i \le j} \end{bmatrix}, \tag{65}$$

με

$$z_{V,k} = \begin{bmatrix} \sin q_{1,k} \\ \cos q_{1,k} \\ \sin q_{2,k} \\ \cos q_{2,k} \\ \tanh\left(\dot{q}_{1,k}/1.5\right) \\ \tanh\left(\dot{q}_{2,k}/1.5\right) \\ v_{1,k}/5 \\ v_{2,k}/5 \\ (N-k+1)/N \end{bmatrix} \tag{66}$$

Ο critic εκτιμά το αναμενόμενο return από το συγκεκριμένο state, την εικονική είσοδο και το χρονικό βήμα. Επομένως, το advantage ορίζεται ως:

$$\hat{A}_k = G_k - V_w(x_k, v_k, k) \tag{67}$$

Πριν από τις επαναλαμβανόμενες ενημερώσεις του actor, τα advantages κανονικοποιούνται μία φορά χρησιμοποιώντας ολόκληρο το batch:

$$\hat{A}_k \leftarrow \frac{\hat{A}_k - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-8}} \tag{68}$$

Ο critic εκπαιδεύεται ξεχωριστά από τον actor, μέσω regression στα πραγματικά returns:

$$w \leftarrow \arg\min_w \left[\frac{1}{M} \sum_{k=1}^{M} \left(V_w(x_k, v_k, k) - G_k \right)^2 + \lambda \|w\|^2 \right] \tag{69}$$

Για το double pendulum, στη συγκεκριμένη περίπτωση, χρησιμοποιούνται 55 παράμετροι w για τον critic.

Για να μετρηθεί πόσο έχει μεταβληθεί η πιθανότητα ενός ήδη δειγματοληπτημένου action, ορίζεται το probability ratio:

$$\rho_k(\theta) = \frac{\pi_\theta\left(u_k \mid x_k, v_k\right)}{\pi_{\theta_{\text{old}}}\left(u_k \mid x_k, v_k\right)} \tag{70}$$

Αν $\rho_k(\theta) > 1$, τότε η νέα πολιτική θεωρεί το συγκεκριμένο action πιθανότερο από ό,τι η παλιά πολιτική, ενώ αν $\rho_k(\theta) < 1$, η νέα πολιτική έχει μειώσει την πιθανότητα επιλογής του συγκεκριμένου action.

Στην αρχή της διαδικασίας έχουμε:

$$\theta = \theta_{\text{old}}, \qquad \rho_k(\theta_{\text{old}}) = 1 \tag{71}$$

Επειδή στην PPO πραγματοποιούνται πολλές ενημερώσεις χρησιμοποιώντας το ίδιο σύνολο δεδομένων, το probability ratio μπορεί να απομακρυνθεί σημαντικά από τη μονάδα, προκαλώντας υπερβολικά μεγάλη μεταβολή της πολιτικής. Για τον λόγο αυτό ορίζεται το clipped probability ratio:

$$\text{clip}\left(\rho_k(\theta), 1-\epsilon, 1+\epsilon\right) \tag{72}$$

όπου ϵ είναι μία υπερπαράμετρος που καθορίζει το εύρος μέσα στο οποίο είναι επιθυμητό να παραμένει η μεταβολή του probability ratio. Εδώ επιλέγεται (όπως και συνήθως):

$$\epsilon = 0.2 \tag{73}$$

επομένως:

$$[1-\epsilon, 1+\epsilon] = [0.8, 1.2] \tag{74}$$

Η PPO χρησιμοποιεί τελικά το clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_k\left[\min\left(\rho_k(\theta)\hat{A}_k, \text{clip}\left(\rho_k(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_k\right)\right] \tag{75}$$

Και έτσι η ενημέρωση των παραμέτρων γίνεται:

$$\theta \leftarrow \theta + \alpha \nabla_\theta L^{\text{CLIP}}(\theta) \tag{76}$$

Το clipping περιορίζει την επίδραση της μεταβολής του probability ratio, χωρίς όμως να εγγυάται ότι η συνολική νέα πολιτική θα παραμείνει αρκετά κοντά στην παλιά. Για τον λόγο αυτό χρησιμοποιείται επιπλέον η KL divergence μεταξύ της παλιάς και της νέας policy.

Έχουμε:

$$D_{\text{KL}}\left(\pi_{\theta_{\text{old}}} \| \pi_\theta\right) = \frac{1}{M} \sum_{k=1}^{M} \frac{\left\| \mu_{\theta_{\text{old}}}(x_k, v_k) - \mu_\theta(x_k, v_k) \right\|^2}{2\sigma^2} \tag{77}$$

Όσο περισσότερο μεταβάλλονται οι μέσες τιμές των δύο Gaussian policies, τόσο αυξάνεται το KL. Χρησιμοποιήθηκε:

$$D_{\text{KL,target}} = 0.02 \tag{78}$$

Αν προκύψει:

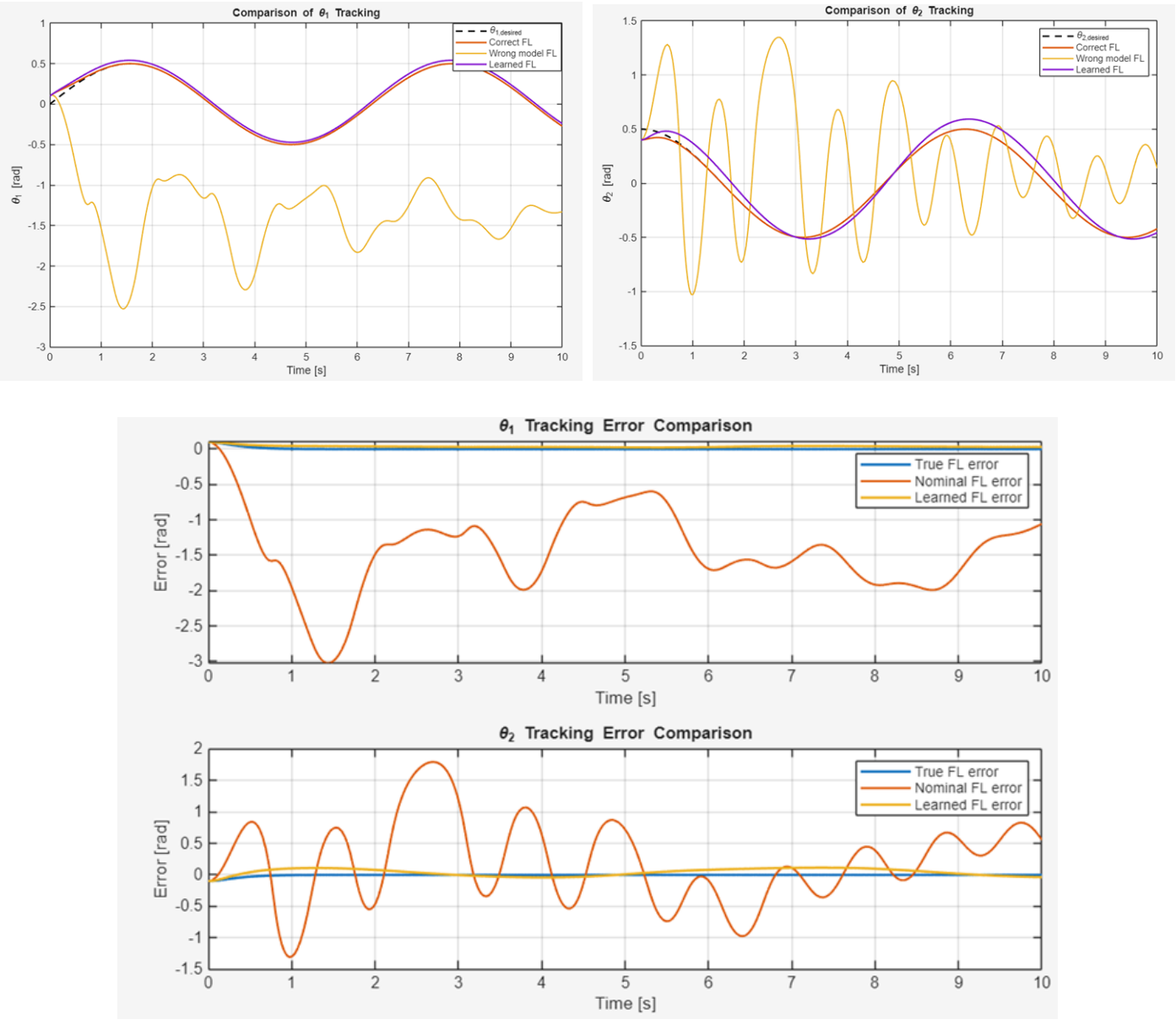
$$D_{\text{KL}} > 1.5 D_{\text{KL,target}} = 0.03 \tag{79}$$

η υποψήφια ενημέρωση απορρίπτεται και διακόπτονται οι υπόλοιπες ενημερώσεις του actor για το συγκεκριμένο iteration. Ο συντελεστής 1.5 χρησιμοποιείται ως ένα μικρό περιθώριο.

Όπως και στη REINFORCE, χρησιμοποιήθηκε:

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \tag{80}$$

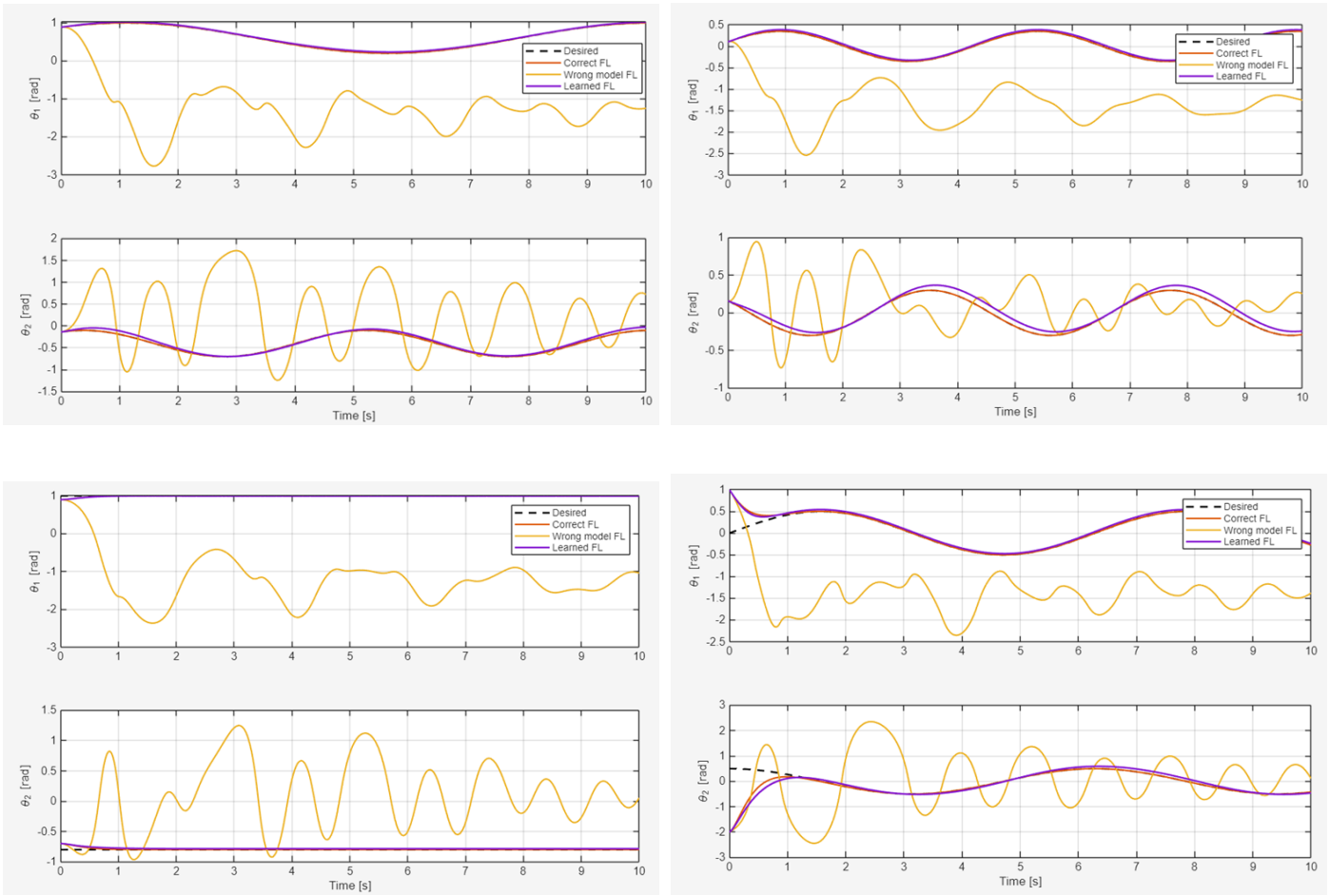
Αποτελέσματα:



Σχήμα 7: Αποτελέσματα PPO για 15000 epochs

Ο controller ανταποκρίθηκε εν γένει πολύ καλά, ενώ φάνηκε πως το θ_1 ανταποκρίθηκε καλύτερα από το θ_2, πιθανώς λόγω της διπλάσιας βαρύτητας που θέσαμε μέσω του μητρώου Q.

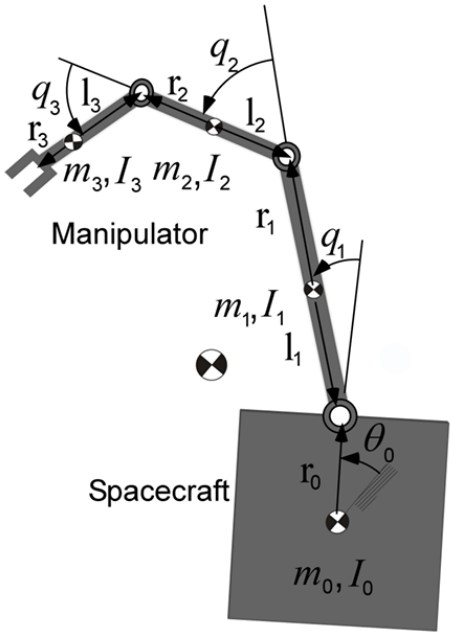
Χρησιμοποίησα και εδώ τον «εκπαιδευμένο» controller για την υλοποίηση των ίδιων trajectories – tasks που χρησιμοποίησα και για την REINFORCE μέθοδο.



Σχήμα 8: Αποτελέσματα PPO για διαφορετικά trajectories – tasks

2 Μοντέλο του Planar 2D Cepheus

Ως δεύτερο σύστημα μελέτης χρησιμοποιήθηκε μία planar 2D εκδοχή του Cepheus, αποτελούμενη από μία διαστημική βάση και έναν manipulator τριών αρθρώσεων και links.



Σχήμα 9: Σχηματική αναπαράσταση του planar 2D Cepheus.

2.1 Κινηματική μοντελοποίηση

Οι απόλυτες γωνίες των links ορίζονται ως:

$$\begin{aligned} \theta_1 &= \theta_0 + q_1, \\ \theta_2 &= \theta_0 + q_1 + q_2, \\ \theta_3 &= \theta_0 + q_1 + q_2 + q_3. \end{aligned} \tag{81}$$

Οι απόλυτες θέσεις των κέντρων μάζας των links είναι:

$$\begin{aligned} \mathbf{p}_1 &= \mathbf{p}_0 + r_0 \begin{bmatrix} \cos\theta_0 \\ \sin\theta_0 \end{bmatrix} + l_1 \begin{bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{bmatrix} \\ \mathbf{p}_2 &= \mathbf{p}_0 + r_0 \begin{bmatrix} \cos\theta_0 \\ \sin\theta_0 \end{bmatrix} + (l_1 + r_1) \begin{bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{bmatrix} + l_2 \begin{bmatrix} \cos\theta_2 \\ \sin\theta_2 \end{bmatrix} \\ \mathbf{p}_3 &= \mathbf{p}_0 + r_0 \begin{bmatrix} \cos\theta_0 \\ \sin\theta_0 \end{bmatrix} + (l_1 + r_1) \begin{bmatrix} \cos\theta_1 \\ \sin\theta_1 \end{bmatrix} + (l_2 + r_2) \begin{bmatrix} \cos\theta_2 \\ \sin\theta_2 \end{bmatrix} + l_3 \begin{bmatrix} \cos\theta_3 \\ \sin\theta_3 \end{bmatrix} \end{aligned} \tag{82}$$

Οι αντίστοιχες γωνιακές ταχύτητες είναι:

$$\begin{aligned}
\omega_0 &= \dot{\theta}_0, \\
\omega_1 &= \dot{\theta}_1 = \dot{\theta}_0 + \dot{q}_1, \\
\omega_2 &= \dot{\theta}_2 = \dot{\theta}_0 + \dot{q}_1 + \dot{q}_2, \\
\omega_3 &= \dot{\theta}_3 = \dot{\theta}_0 + \dot{q}_1 + \dot{q}_2 + \dot{q}_3.
\end{aligned} \tag{83}$$

Η γραμμική ταχύτητα της βάσης δίνεται από:

$$\mathbf{v}_0 = \dot{\mathbf{p}}_0 \tag{84}$$

Οι γραμμικές ταχύτητες των κέντρων μάζας των links προκύπτουν παραγωγίζοντας τις αντίστοιχες θέσεις:

$$\begin{aligned}
\mathbf{v}_1 &= \mathbf{v}_0 + r_0\dot{\theta}_0 \begin{bmatrix} -\sin\theta_0 \\ \cos\theta_0 \end{bmatrix} + l_1\dot{\theta}_1 \begin{bmatrix} -\sin\theta_1 \\ \cos\theta_1 \end{bmatrix} \\
\mathbf{v}_2 &= \mathbf{v}_0 + r_0\dot{\theta}_0 \begin{bmatrix} -\sin\theta_0 \\ \cos\theta_0 \end{bmatrix} + (l_1 + r_1)\dot{\theta}_1 \begin{bmatrix} -\sin\theta_1 \\ \cos\theta_1 \end{bmatrix} + l_2\dot{\theta}_2 \begin{bmatrix} -\sin\theta_2 \\ \cos\theta_2 \end{bmatrix} \\
\mathbf{v}_3 &= \mathbf{v}_0 + r_0\dot{\theta}_0 \begin{bmatrix} -\sin\theta_0 \\ \cos\theta_0 \end{bmatrix} + (l_1 + r_1)\dot{\theta}_1 \begin{bmatrix} -\sin\theta_1 \\ \cos\theta_1 \end{bmatrix} + (l_2 + r_2)\dot{\theta}_2 \begin{bmatrix} -\sin\theta_2 \\ \cos\theta_2 \end{bmatrix} + l_3\dot{\theta}_3 \begin{bmatrix} -\sin\theta_3 \\ \cos\theta_3 \end{bmatrix}
\end{aligned} \tag{85}$$

Έχουμε τις γενικευμένες συντεταγμένες:

$$\xi = \begin{bmatrix} \theta_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}, \qquad \dot{\xi} = \begin{bmatrix} \dot{\theta}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} \tag{86}$$

Λαμβάνοντας υπόψη τη διατήρηση της γραμμικής ορμής:

$$m_0\mathbf{v}_0 + m_1\mathbf{v}_1 + m_2\mathbf{v}_2 + m_3\mathbf{v}_3 = 0 \tag{87}$$

Οι ταχύτητες των links μπορούν να γραφούν ως:

$$\mathbf{v}_1 = \mathbf{v}_0 + \mathbf{v}_{1,\text{rel}}, \qquad \mathbf{v}_2 = \mathbf{v}_0 + \mathbf{v}_{2,\text{rel}}, \qquad \mathbf{v}_3 = \mathbf{v}_0 + \mathbf{v}_{3,\text{rel}} \tag{88}$$

Επομένως, προκύπτει μία σχέση για τη γραμμική ταχύτητα της βάσης που τη συνδέει με τις γενικευμένες συντεταγμένες και ταχύτητες:

$$\mathbf{v}_0 = -\frac{m_1\mathbf{v}_{1,\text{rel}} + m_2\mathbf{v}_{2,\text{rel}} + m_3\mathbf{v}_{3,\text{rel}}}{m_0 + m_1 + m_2 + m_3} \tag{89}$$

Η κινητική ενέργεια κάθε σώματος του συστήματος υπολογίζεται ως:

$$T_i = \frac{1}{2}m_i\mathbf{v}_i^T\mathbf{v}_i + \frac{1}{2}I_i\omega_i^2, \qquad i = 0, 1, 2, 3 \tag{90}$$

Η συνολική κινητική ενέργεια του συστήματος είναι:

$$T = T_0 + T_1 + T_2 + T_3 \tag{91}$$

Θεωρώντας μηδενική βαρύτητα, η δυναμική ενέργεια είναι $V = 0$, οπότε η Lagrangian του συστήματος γράφεται:

$$L = T - V = T \tag{92}$$

2.2 Δυναμική μοντελοποίηση

Μέσω των εξισώσεων Euler–Lagrange σχηματίζεται το δυναμικό μοντέλο του συστήματος:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\xi}}\right) - \frac{\partial L}{\partial \xi} = \tau \tag{93}$$

και προκύπτει η τελική δυναμική μορφή:

$$H(\xi)\ddot{\xi} + c(\xi, \dot{\xi}) = \tau \tag{94}$$

όπου:

$$H(\xi) \in \mathbb{R}^{4\times 4}, \qquad c(\xi, \dot{\xi}) \in \mathbb{R}^{4\times 1} \tag{95}$$

Το διάνυσμα εισόδου είναι:

$$\tau = \begin{bmatrix} \tau_{\mathrm{rw}} \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix} \tag{96}$$

Το διάνυσμα κατάστασης ορίζεται ως:

$$x = \begin{bmatrix} \xi \\ \dot{\xi} \end{bmatrix} = \begin{bmatrix} \theta_0 \\ q_1 \\ q_2 \\ q_3 \\ \dot{\theta}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} \tag{97}$$

Από τη δυναμική εξίσωση έχουμε:

$$\ddot{\xi} = H(\xi)^{-1}\left[\tau - c(\xi, \dot{\xi})\right] \tag{98}$$

Επομένως, το σύστημα γράφεται σε state-space μορφή ως:

$$\dot{x} = \begin{bmatrix} \dot{\xi} \\ H(\xi)^{-1}\left[\tau - c(\xi, \dot{\xi})\right] \end{bmatrix} \tag{99}$$

2.3 Feedback Linearization

Βάσει της μεθόδου του Westenbroek, για το πραγματικό μοντέλο εφαρμόζεται feedback linearization επιλέγοντας την είσοδο:

$$\tau = \beta_m + \alpha_m v = c(\xi, \dot{\xi}) + H(\xi)v \tag{100}$$

με αποτέλεσμα:

$$\ddot{\xi} = v \tag{101}$$

Η εικονική είσοδος επιλέγεται ως:

$$v = \ddot{\xi}_d - K_d\big(\dot{\xi} - \dot{\xi}_d\big) - K_p\big(\xi - \xi_d\big) \tag{102}$$

Ορίζοντας:

$$e = \xi - \xi_d, \qquad \dot{e} = \dot{\xi} - \dot{\xi}_d \tag{103}$$

προκύπτει η δυναμική του σφάλματος:

$$\ddot{e} + K_d\dot{e} + K_p e = 0 \tag{104}$$

Στο nominal μοντέλο τα δυναμικά μεγέθη του συστήματος συμβολίζονται ως:

$$\beta_m = \hat{c}(\xi, \dot{\xi}), \qquad \alpha_m = \hat{H}(\xi) \tag{105}$$

και ο nominal feedback linearizing controller είναι:

$$\tau_{\text{nom}} = \beta_m + \alpha_m v \tag{106}$$

Έχουμε:

$$\tau = [\beta_m + \beta_\theta] + [\alpha_m + \alpha_\theta]\, v \tag{107}$$

Όμοια με την περίπτωση του double pendulum, οι διορθωτικοί όροι παραμετροποιούνται ως:

$$\beta_\theta = \Theta_\beta \phi_\beta(x), \qquad \alpha_\theta = \Theta_\alpha \phi_\alpha(x) \tag{108}$$

Τα nonlinear features του ϕ_α αποτελούνται κυρίως από συνημιτονοειδείς συναρτήσεις των σχετικών γωνιών και των αθροισμάτων τους, καθώς αντίστοιχοι όροι εμφανίζονται στο μητρώο $H(\xi)$.

Αντίστοιχα, τα nonlinear features του ϕ_β περιλαμβάνουν ημιτονοειδείς συναρτήσεις των ίδιων γωνιακών συνδυασμών, καθώς και τετραγωνικά και μικτά γινόμενα των γενικευμένων ταχυτήτων, όπως εμφανίζονται στον όρο $c(\xi, \dot{\xi})$.

Παράμετροι πραγματικού μοντέλου

Για το πραγματικό μοντέλο του συστήματος χρησιμοποιήθηκαν:

Spacecraft:

$$m_0 = 54.00\,\text{kg}, \qquad I_0 = 2.1837\,\text{kg}\,\text{m}^2, \qquad r_0 = 0.1645\,\text{m}$$

Link 1:

$$m_1 = 0.23\,\text{kg}, \qquad I_1 = 6.81 \times 10^{-3}\,\text{kg}\,\text{m}^2, \qquad l_1 = r_1 = 0.185\,\text{m}$$

Link 2:

$$m_2 = 0.10\,\text{kg}, \qquad I_2 = 1.487 \times 10^{-5}\,\text{kg}\,\text{m}^2, \qquad l_2 = r_2 = 0.143\,\text{m}$$

Link 3:

$$m_3 = 2.50\,\text{kg}, \qquad I_3 = 6.5262 \times 10^{-2}\,\text{kg}\,\text{m}^2, \qquad l_3 = 0.0411\,\text{m}, \qquad r_3 = 0.0261\,\text{m}$$

2.4 Nominal μοντέλο

Χρησιμοποιήθηκε ένα λανθασμένο-nominal μοντέλο με έντονες αποκλίσεις στις παραμέτρους:

Spacecraft:

$$\hat{m}_0 = 0.3m_0, \qquad \hat{r}_0 = 1.6r_0 \tag{109}$$

Link 1:

$$\hat{m}_1 = 1.7m_1, \qquad \hat{l}_1 = 0.6l_1, \qquad \hat{r}_1 = 0.7r_1 \tag{110}$$

Link 2:

$$\hat{m}_2 = 0.5m_2, \qquad \hat{l}_2 = 1.6l_2, \qquad \hat{r}_2 = 1.7r_2 \tag{111}$$

Link 3:

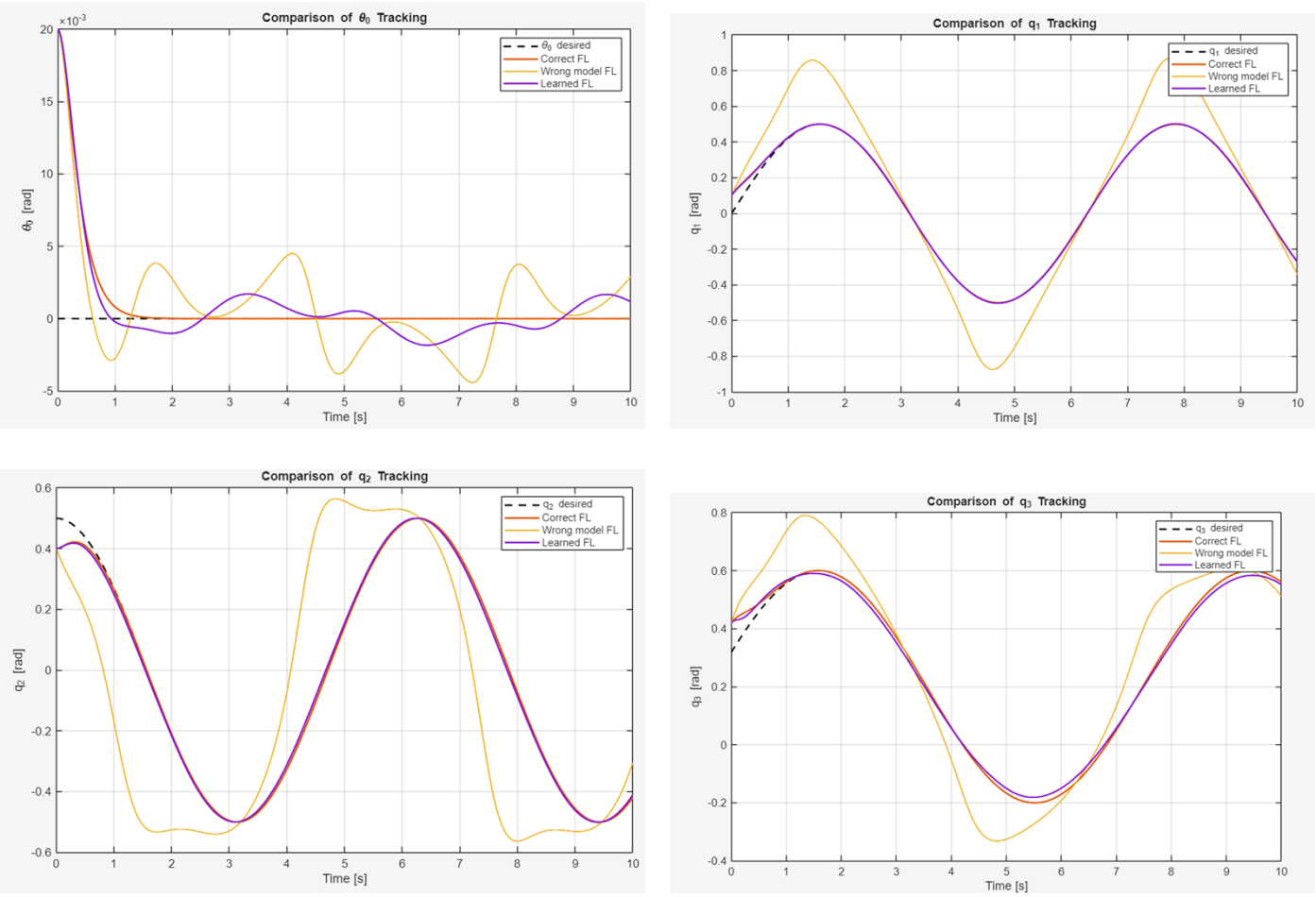
$$\hat{m}_3 = 1.8m_3, \qquad \hat{l}_3 = 0.7l_3, \qquad \hat{r}_3 = 0.6r_3 \tag{112}$$

Οι ροπές αδράνειας του nominal μοντέλου επειδή εξαρτώνται από το γινόμενο της μάζας και του τετραγώνου του μήκους υπολογίζονται προσεγγιστικά:

$$\hat{I}_i = I_i \left(\frac{\hat{m}_i}{m_i}\right) \left(\frac{\hat{L}_i}{L_i}\right)^2, \qquad i = 0, 1, 2, 3 \tag{113}$$

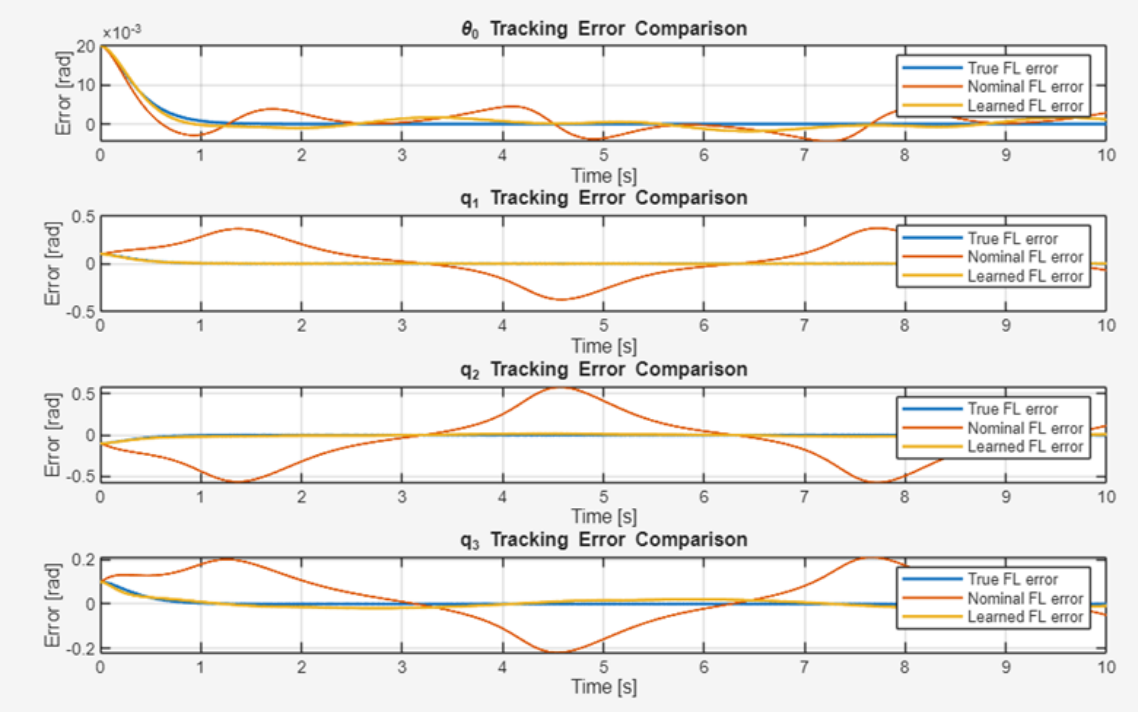
2.5 REINFORCE

Εφαρμόστηκε η μέθοδος REINFORCE με διαφορετική λογική από αυτή που χρησιμοποιήθηκε στο double pendulum, καθώς φάνηκε πως για να επιτευχθεί σύγκλιση στο σύστημα όταν οι αποκλίσεις του nominal μοντέλου ήταν πολύ μεγάλες, ήταν αποτελεσματική η υλοποίηση **ενός χρονικού βήματος σε κάθε rollout**, όπως υλοποιήθηκε και στο Baxter στο paper του Westenbroek. Έτσι σε κάθε epoch πραγματοποιούνταν **1000 rollouts** και στο καθένα από αυτά **ένα χρονικό βήμα $\Delta t = 0.01$ s**. Έτσι προέκυψαν τα ακόλουθα αποτελέσματα μετά από **2000 epochs**:



Σχήμα 10: Αποτελέσματα REINFORCE για το βασικό test του Planar 2D Cepheus

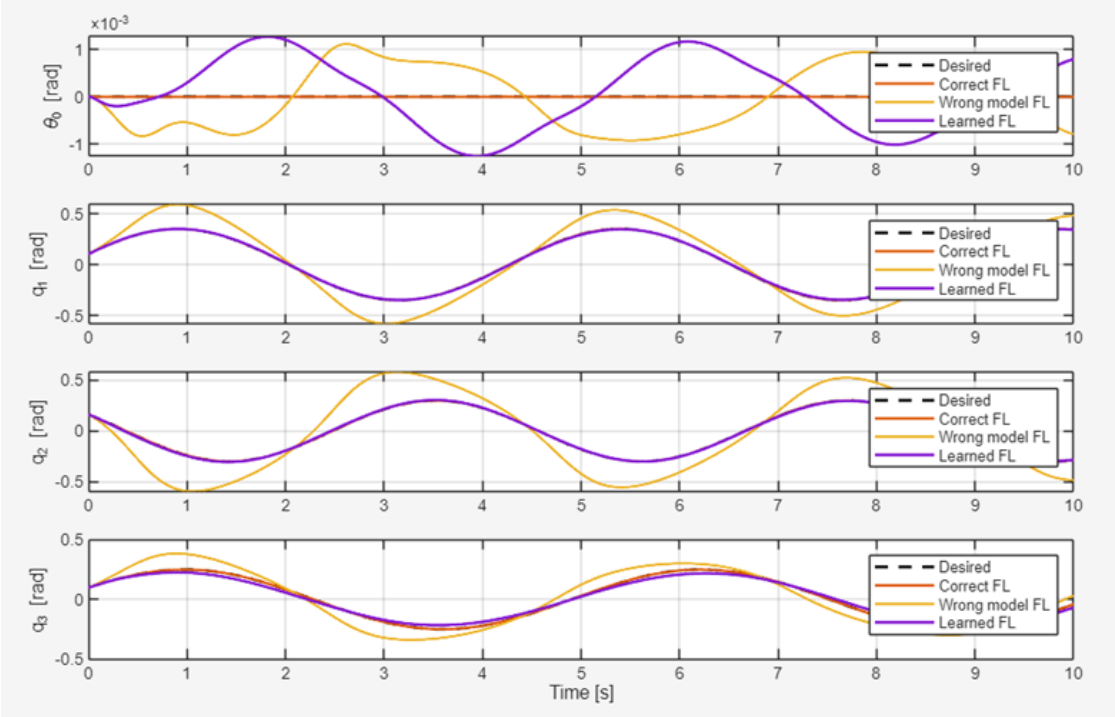
Τα αντίστοιχα σφάλματα του tracking παρουσιάζονται στο ακόλουθο διάγραμμα:



Σχήμα 11: Tracking Errors για την REINFORCE.

Για:

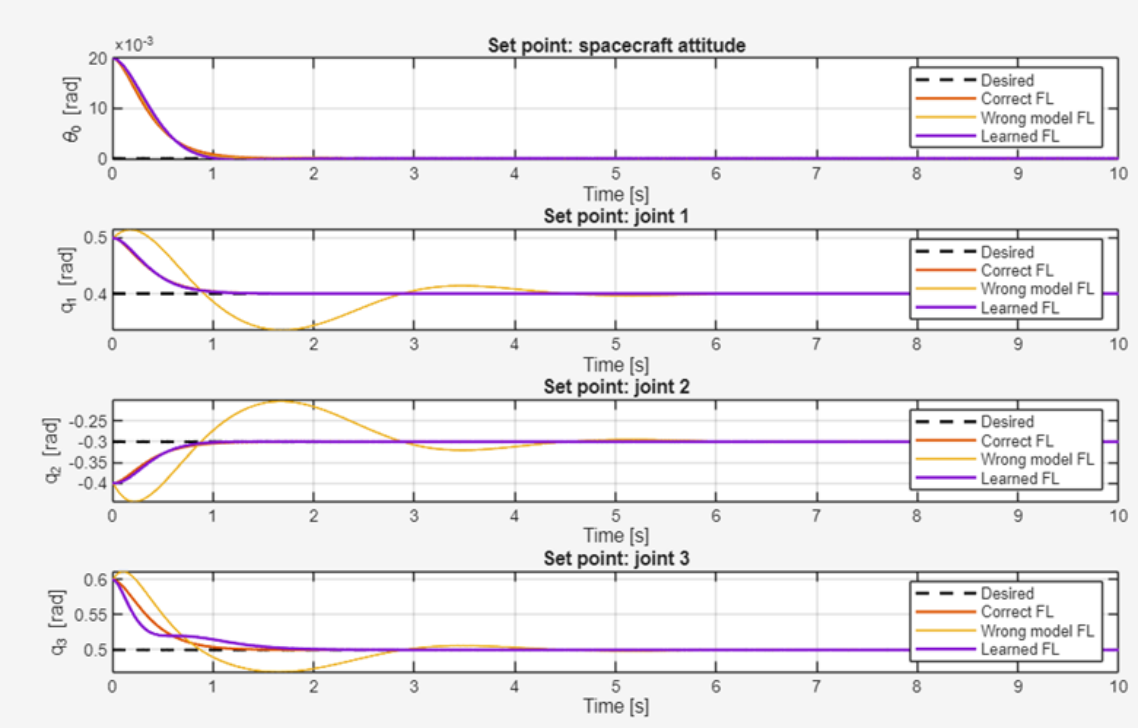
$$
\begin{aligned}
\theta_{0,d}(t) &= 0, \\
q_{1,d}(t) &= 0.15 + 0.35\,\sin(0.65t + 0.40), \\
q_{2,d}(t) &= -0.10 + 0.45\,\sin\left(1.35t + \frac{\pi}{2} - 0.70\right), \\
q_{3,d}(t) &= 0.25 + 0.30\,\sin(0.90t + 0.50).
\end{aligned}
\tag{114}
$$



Σχήμα 12: Αποτελέσματα REINFORCE για διαφορετικό scenario

Για:

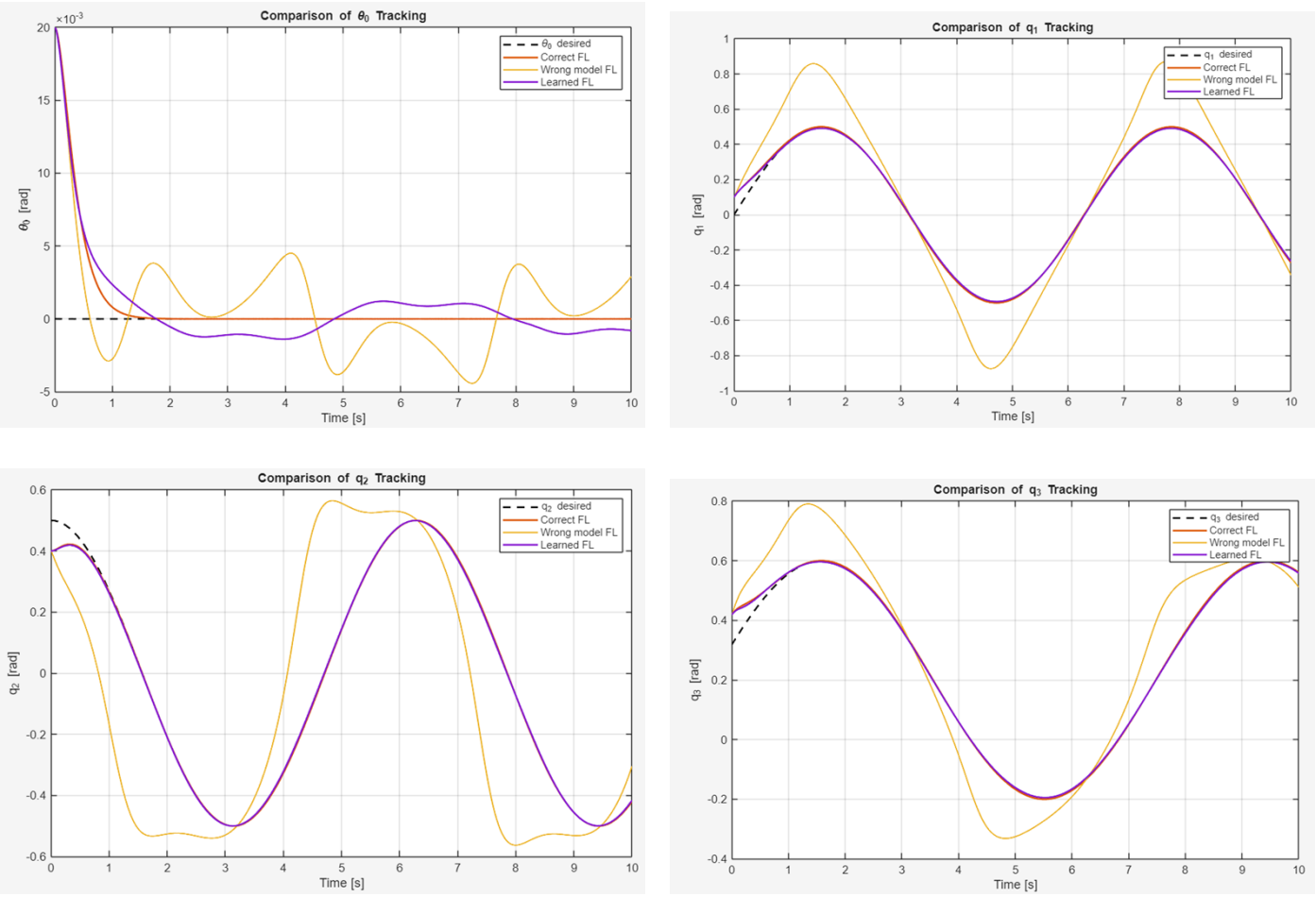
$$
\begin{aligned}
\theta_{0,d}(t) &= 0, \\
q_{1,d}(t) &= 0.35\sin(1.40t + 0.30), \\
q_{2,d}(t) &= 0.30\sin\left(1.50t + \frac{\pi}{2} + 1.00\right), \\
q_{3,d}(t) &= 0.25\sin(1.20t + 0.40).
\end{aligned}
\tag{115}
$$



Σχήμα 13: Αποτελέσματα REINFORCE για διαφορετικό scenario

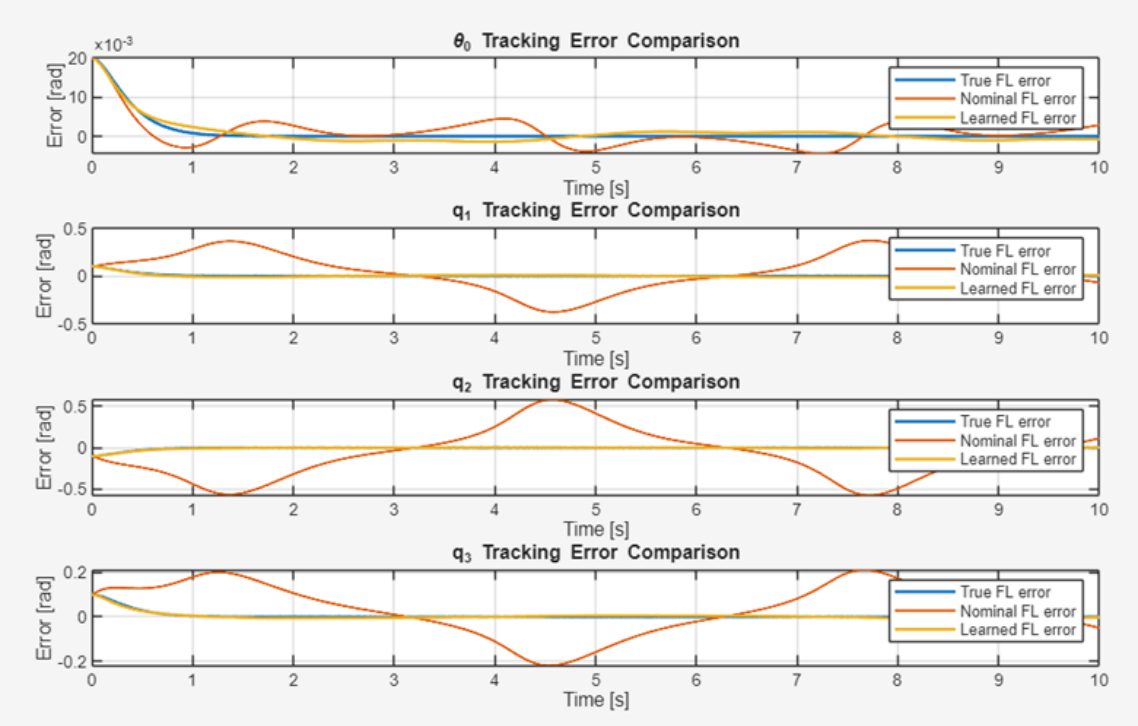
2.6 PPO

Αντίστοιχα, εφαρμόστηκε ο αλγόριθμος PPO για την εκπαίδευση του controller. Πάλι με την υλοποίηση **ενός χρονικού βήματος σε κάθε rollout** και για **1250 rollouts** σε κάθε iteration, τα αποτελέσματα μετά από **1000 iterations** ήταν:



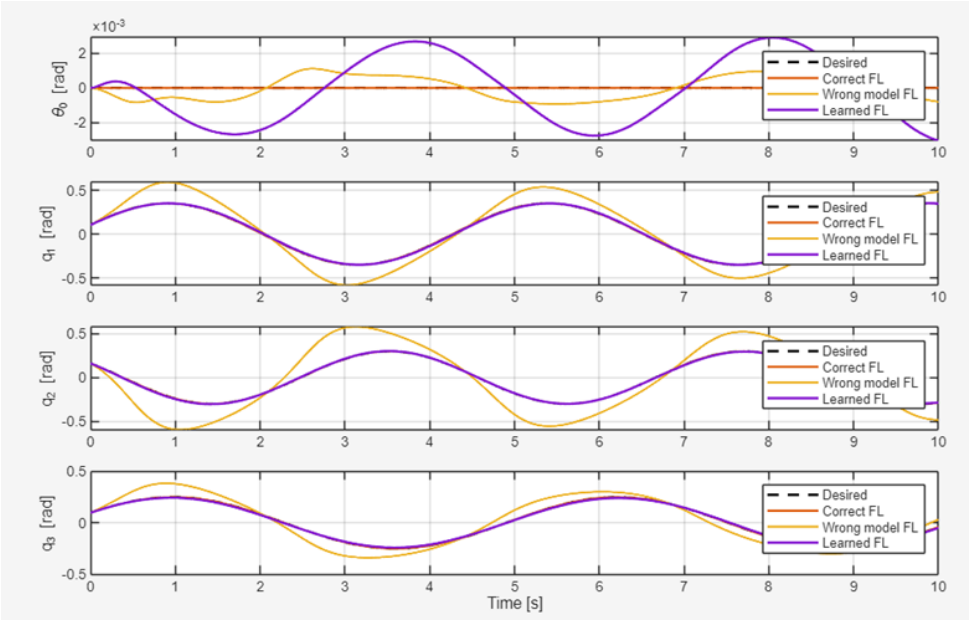
Σχήμα 14: Αποτελέσματα PPO

Τα αντίστοιχα σφάλματα:



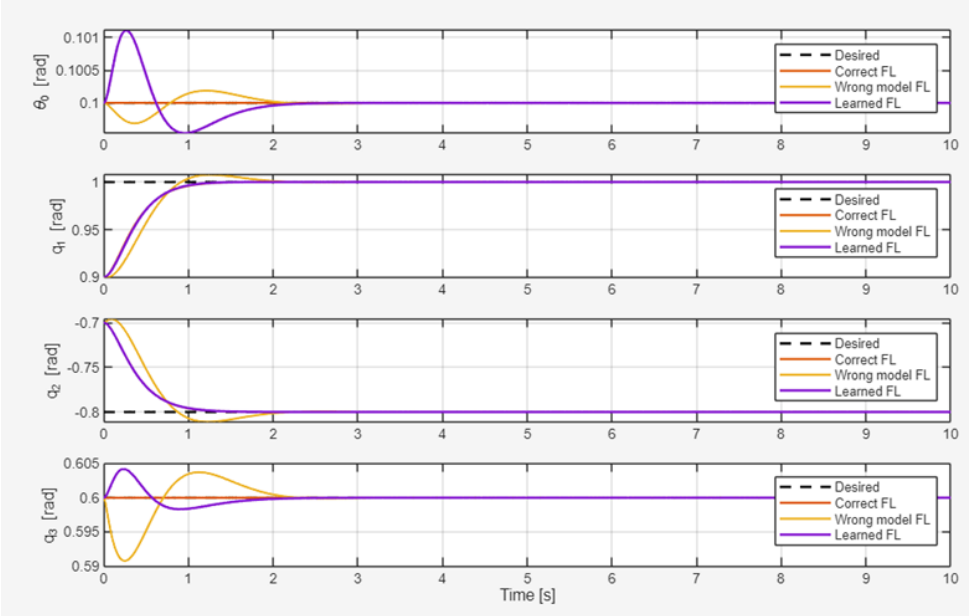
Σχήμα 15: Tracking Errors για τη μέθοδο PPO

Για ημιτονοειδές trajectory:



Σχήμα 16: Αποτελέσματα PPO για διαφορετικό ημιτονοειδές trajectory

Για set point:



Σχήμα 17: Αποτελέσματα PPO για set point

3 Σύγκριση με το σενάριο του paper MED 2026

Η μέθοδος που αναπτύχθηκε προηγουμένως εφαρμόστηκε και στο σενάριο προσομοίωσης που παρουσιάζεται στο paper των Skandalakis et al. (MED 2026). Στόχος ήταν η παρακολούθηση μιας επιθυμητής τροχιάς από τις τρεις αρθρώσεις του βραχίονα, με ταυτόχρονη διατήρηση της γωνίας της βάσης στο μηδέν.

Οι αρχικές και τελικές επιθυμητές γωνίες των αρθρώσεων ήταν:

$$q_d^{in} = \begin{bmatrix} -20^\circ \\ 65^\circ \\ 25^\circ \end{bmatrix}, \qquad q_d^{fin} = \begin{bmatrix} 80^\circ \\ 100^\circ \\ 70^\circ \end{bmatrix}. \tag{116}$$

Η επιθυμητή τροχιά που έπρεπε να ακολουθήσει το σύστημα περιγράφηκε από πολυώνυμο πέμπτου βαθμού, με μηδενικές αρχικές και τελικές επιθυμητές ταχύτητες και επιταχύνσεις, ενώ οι πραγματικές αρχικές γωνίες ήταν διαφορετικές από τις επιθυμητές και ειδικότερα:

$$q(0) = \begin{bmatrix} -15^\circ \\ 60^\circ \\ 30^\circ \end{bmatrix} \tag{117}$$

Στο πραγματικό μοντέλο προσθέθηκε ιξώδης απόσβεση (damping) στις αρθρώσεις με συντελέστη $b = 0.9\,\mathrm{N\,m\,s/rad}$, ενώ στο nominal μοντέλο η απόσβεση θεωρήθηκε μηδενική. Η αβεβαιότητα του fuel sloshing και των flexible panels δεν λήφθηκε υπόψη. Επιπλέον λήφθηκαν οι εξής περιορισμοί στις ροπές:

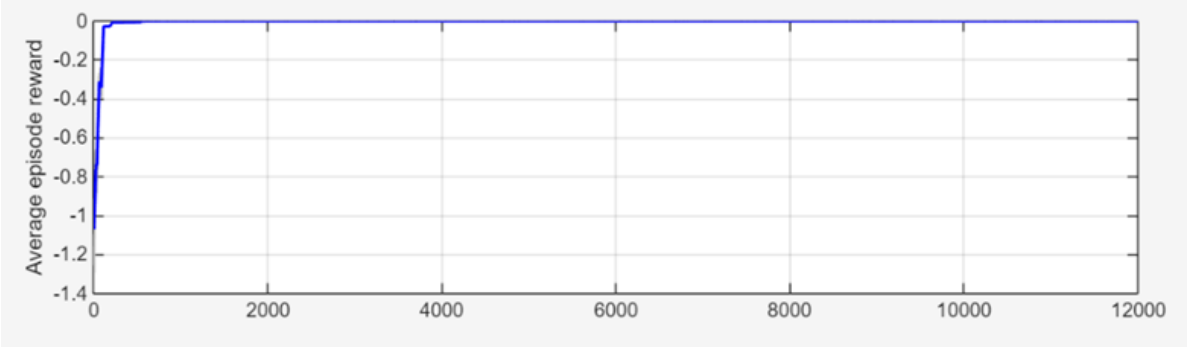
$$|u_0| \le 0.1\,\mathrm{Nm}, \qquad |u_i| \le 0.5\,\mathrm{Nm}, \quad i = 1, 2, 3. \tag{118}$$

Οι περιορισμοί αυτοί εφαρμόστηκαν στον κώδικα, απλώς μέσω clipping των υπολογιζόμενων ροπών ελέγχου στα αντίστοιχα επιτρεπόμενα όρια. Παρ' όλα αυτά όπως φάνηκε δεν έφτασαν κοντά στα όρια αυτά, πιθανώς εξαιτίας του γεγονότος ότι η επιθυμητή τροχιά έπρεπε να πραγματοποιηθεί σε μεγάλο χρονικό διάστημα και συγκεκριμένα σε 50s.

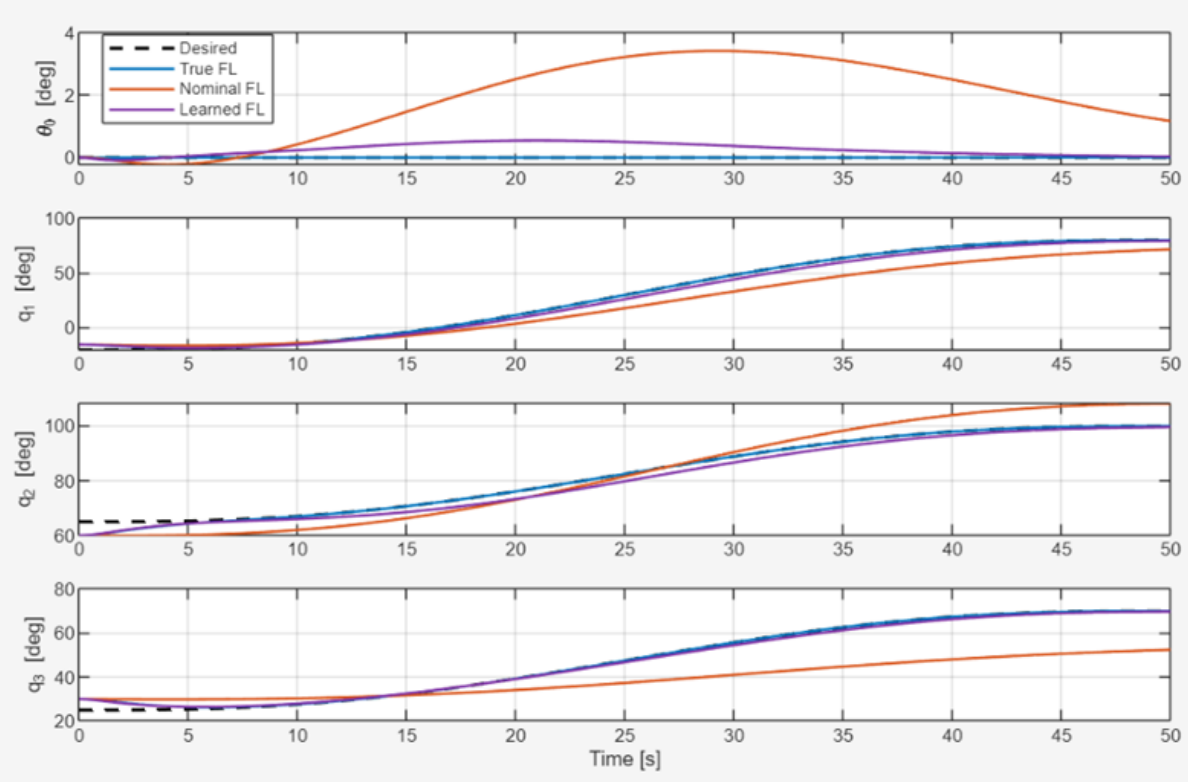
Συγκρίθηκαν λοιπόν πάλι το ιδανικό Feedback Linearization (True FL), το Feedback Linearization με χρήση του nominal μοντέλου (Nominal FL) και το Learned Feedback Linearization, με διορθωτικούς όρους που εκπαιδεύτηκαν μέσω REINFORCE και PPO. Εν τέλει, τα αποτελέσματα εξετάστηκαν σε σχέση με εκείνα που παρουσιάστηκαν στο MED 2026.

3.1 REINFORCE

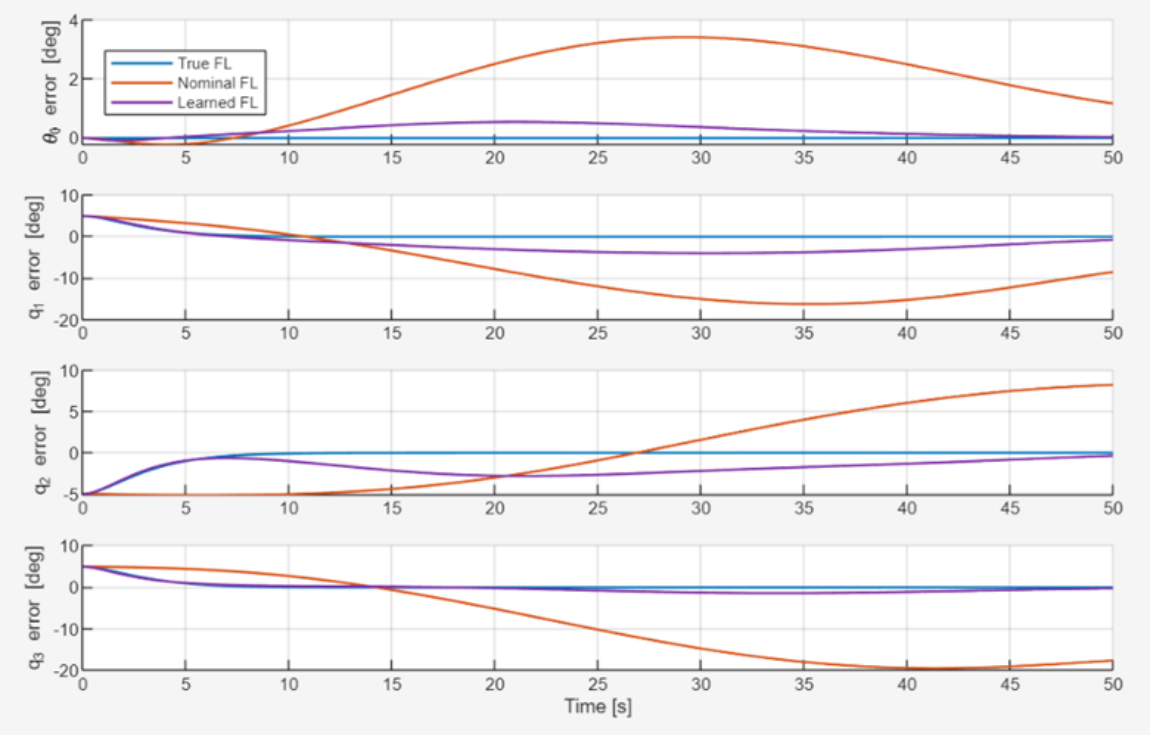
Αρχικά, εφαρμόστηκε ο αλγόριθμος REINFORCE για την εκπαίδευση των διορθωτικών όρων. Χρησιμοποιήθηκε ένα χρονικό βήμα ανά rollout, 100 rollouts ανά epoch και συνολικά 12000 epochs.



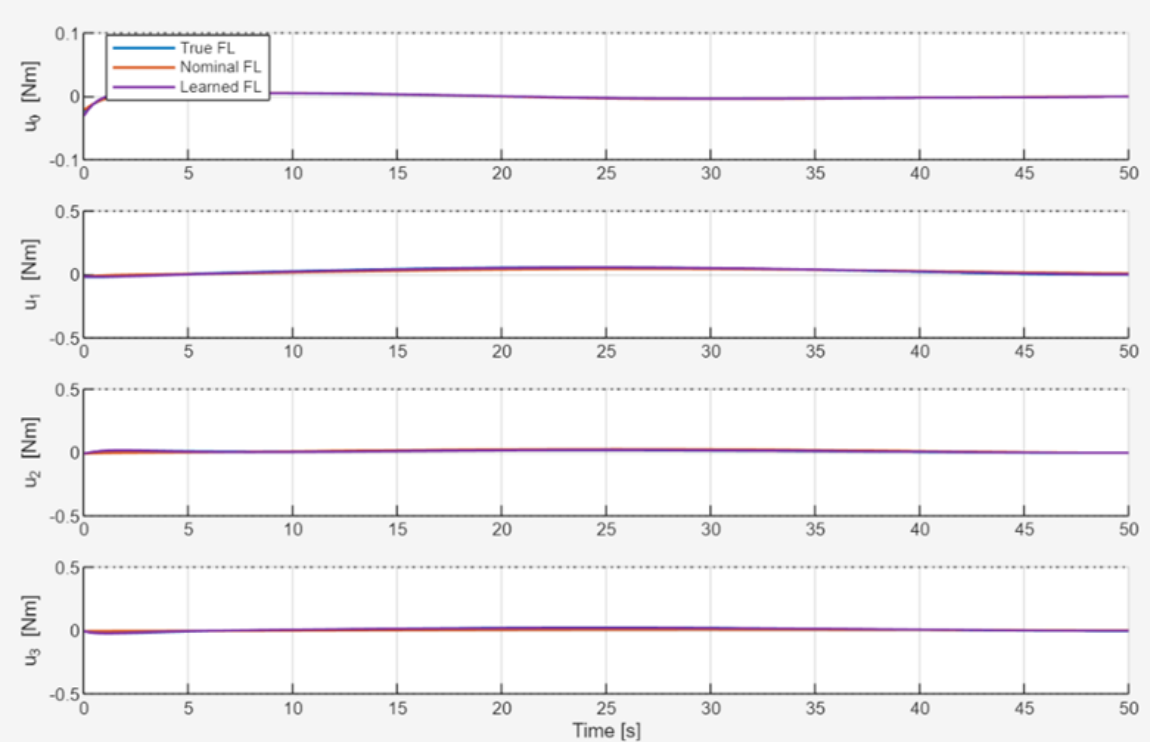
Σχήμα 18: Average Episodic Reward κατά την εκπαίδευση με REINFORCE



Σχήμα 19: Παρακολούθηση της επιθυμητής τροχιάς με REINFORCE



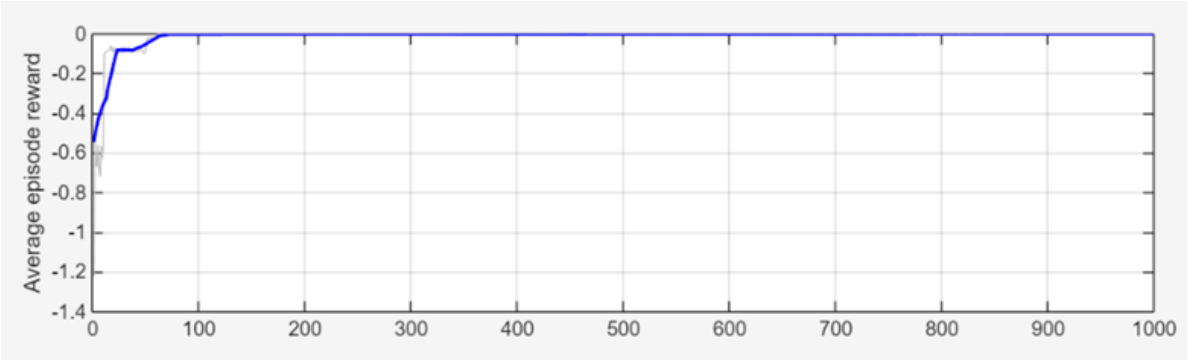
Σχήμα 20: Σφάλματα παρακολούθησης τροχιάς για REINFORCE



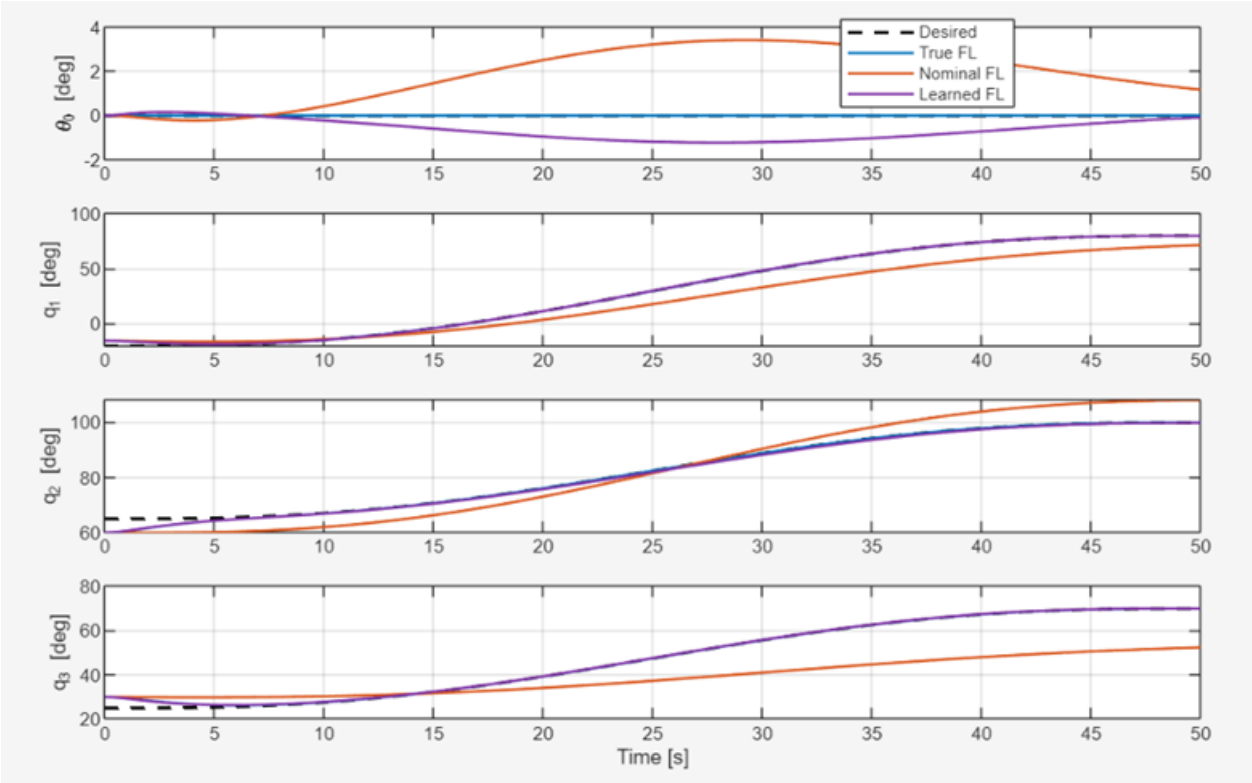
Σχήμα 21: Ροπές ελέγχου

3.2 PPO

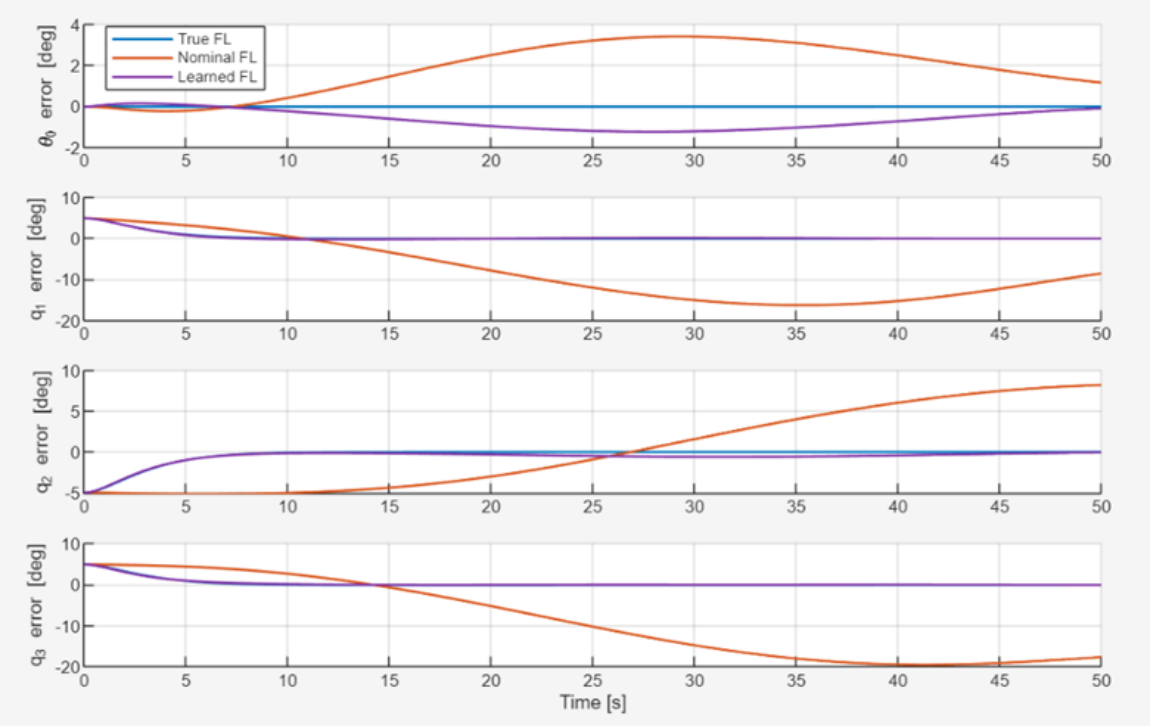
Αντίστοιχα, εφαρμόστηκε ο αλγόριθμος PPO για την εκπαίδευση των διορθωτικών όρων. Χρησιμοποιήθηκε πάλι ένα χρονικό βήμα ανά rollout, 200 rollouts ανά iteration και συνολικά 1000 iterations.



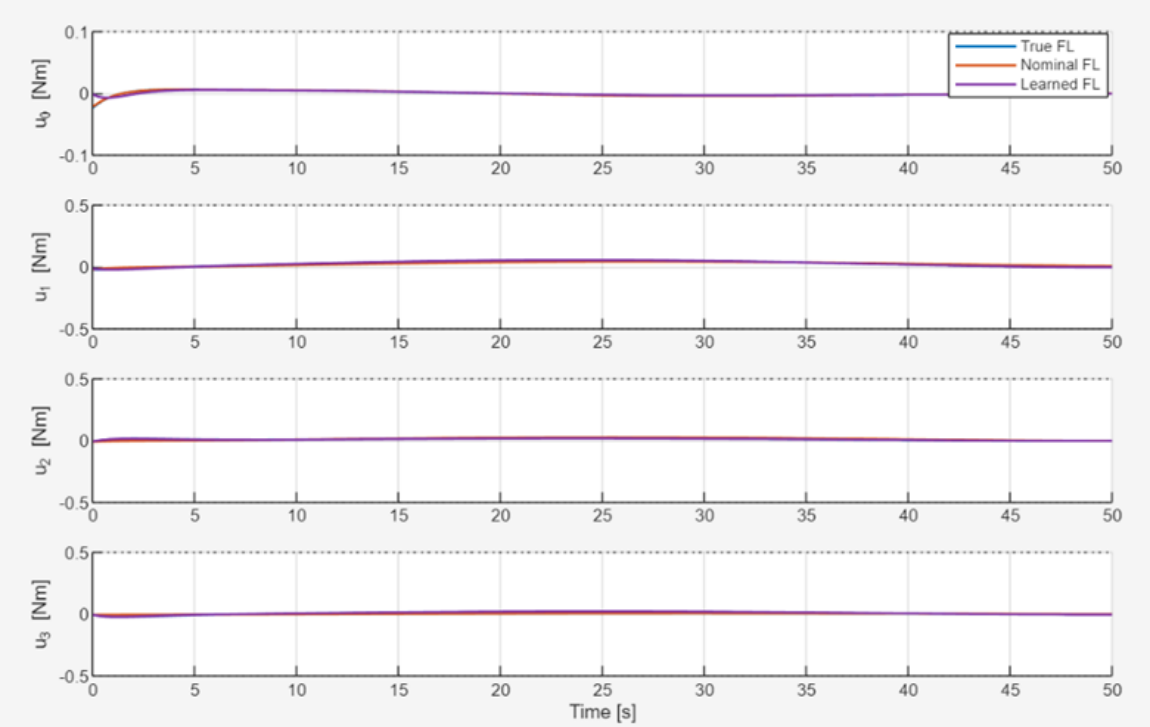
Σχήμα 22: Average Episodic Reward κατά την εκπαίδευση με PPO



Σχήμα 23: Παρακολούθηση της επιθυμητής τροχιάς με PPO



Σχήμα 24: Σφάλματα παρακολούθησης τροχιάς για PPO



Σχήμα 25: Ροπές ελέγχου

Σχολιασμός αποτελεσμάτων και Συμπεράσματα

Στην εργασία αυτή υλοποίηθηκε η μέθοδος που παρουσιάστηκε στο paper των Westenbroek et al. [2], αρχικά για ένα απλό double pendulum, ώστε να φανεί η μεθοδολογία και έπειτα για τον Planar 2D Cepheus, ο οποίος χρησιμοποίηθηκε ως ένα πολυπλοκότερο δυναμικό σύστημα.

Η σύγκλιση των αλγοριθμών RL ήταν αρκετά ευκολότερη κατά την ενασχόληση με το double pendulum. Παρ' όλα αυτά, θεωρήθηκε πως έπρεπε να δοθεί διπλάσια βαρύτητα στο linearization error της πρώτης άρθρωσης κατά την εκπαίδευση μιας και η καμπύλη Learned FL παρουσίαζε γενικώς έντονο offset σε σχέση με αυτή του πραγματικού μοντέλου (True FL). Στον Cepheus από την άλλη πλευρά, για μεγάλες αποκλίσεις του nominal μοντέλου ήταν δύσκολη η σύγκλιση. Παρ' όλα αυτά, όταν χρησιμοποιήθηκε ένα μόνο χρονικό βήμα ανά rollout και περισσότερα rollouts σε κάθε epoch, η σύγκλιση σε όρους epochs/iterations ήταν αρκετά ταχύτερη και μπορούσε να επιτευχθεί ακόμα και για μεγάλες αποκλίσεις από το πραγματικό μοντέλο. Στο σύστημα του Cepheus ο βαθμός ελευθερίας στη βάση ήταν ο μόνος που παρουσίαζε γενικά μη ικανοποιητική συμπεριφορά και τη μεγαλύτερη δυσκολία στη παρακολούθηση, ώστοσο οι αποκλίσεις που παρουσίαζε στις γωνίες ήταν ιδιαίτερα μικρές.

Κατά την υλοποίηση του ίδιου task με αυτό που παρουσιάστηκε στο [5], φάνηκε πως επιτυγχάνεται η εκπαίδευση, όμως η παρακολούθηση τροχιάς δεν πραγματοποιείται με την ίδια ακρίβεια σε σχέση με τον L_1 adaptive controller του συγκεκριμένου paper. Ενώ στον τελευταίο οι αποκλίσεις από την επιθυμητή τροχία ήταν σχεδόν μηδαμινές, με τη χρήση RL παρατηρήθηκαν αποκλίσεις περίπου 0.5° έως 4°, μετά από εκπαίδευση ενός επαρκή και όπως θεωρήθηκε λογικού αριθμού epoch. Η συμπεριφορά αυτή ωστόσο παρουσιάστηκε και στα διαγράμματα του quadrotor στο paper του Westenbroek et al. [2], όπου ήταν εμφανής μία μικρή απόκλιση από την επιθυμητή τροχιά στον 3D χώρο.

Γενικά για την ικανοποιητική εκπαίδευση απαιτήθηκαν αρκετά epochs και γενικότερα πιθανώς μεγαλύτερο υπολογιστικό κόστος με βάση τα νούμερα που παρουσιάστηκαν στο paper του Westenbroek. Επιπροσθέτως, όπως και ήταν αναμενόμμενο, η μέθοδος PPO παρουσίασε γενικά καλύτερα αποτελέσματα από ότι η REINFORCE, κάτι που φάνηκε και από τις τροχιές που αναπαραστάθηκαν.

Το πλεονέκτημα που πιθανώς παρουσιάζει η μέθοδος RL σε σχέση με αυτή του L_1 adaptive controller, ενδεχομένως έγγυται στο γεγονός ότι μπορεί να χρησιμοποιηθεί σε συστήματα όπου η αποκλίσεις του nominal μοντέλου από το πραγματικό και γενικότερα οι αβεβαιότητες είναι πολύ μεγάλες, παρ' όλα αυτά δεν παρουσιάζει την ίδια ακρίβεια με την τελευταία.

Βιβλιογραφία

[1] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Englewood Cliffs, NJ: Prentice Hall, 1991, ch. 6, pp. 211–212.

[2] T. Westenbroek, D. Fridovich-Keil, E. Mazumdar, S. Arora, V. Prabhu, S. S. Sastry, and C. J. Tomlin, "Feedback Linearization for Uncertain Systems via Reinforcement Learning," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 1364–1371.

[3] J. Peters and S. Schaal, "Policy Gradient Methods for Robotics," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2006, pp. 2219–2225.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[5] N. Skandalakis, K. Nanos, and I. Poulakakis, "An L1-Adaptive Control Lyapunov Function Quadratic Program for Torque Constrained Space Manipulators under Uncertainty," in *Proc. 34th Mediterranean Conference on Control and Automation (MED)*, 2026, pp. 698–703.